Alma Mater Studiorum · Università di Bologna

Second Cycle Degree
Artificial Intelligence

Combinatorial Decision Making and Optimization

Student:
Matteo Canghiari

Academic Year 2025/2026

Chapter 1

Introduction

Chapter 2

Modelling in CP

We need to start from the **model**: it doesn't just give us a formal representation of our problem, but also the modelling choices directly impact the search performance.

2.1 Formalization as a Constraint Satisfaction Problem (CSP)

A formal definition of any **Constraint Satisfaction Problem** (*CSP*) can be described as follows.

Definition

A CSP is a triple $\langle \mathbf{X}, \mathbf{D}, \mathbf{C} \rangle$ where:

- **X** is a set of decision variables $\{X_1, \ldots, X_n\}$, which we have to associate a value.
- **D** is a set of domains $\{D_1, \ldots, D_n\}$, where D_i is the domain of the decision variable X_i with $i \in \{1, \ldots, n\}$.
- **C** is a set of constraints $\{C_1, \ldots, C_m\}$. Every constraint C_i with $i \in \{1, \ldots, m\}$ is a mathematical relation over a subset of variables $\{X_j, \ldots, X_k\}$, denoted as $C_i\{X_j, \ldots, X_k\}$. Being a relation means a subset of the cartesian product of all the domains of the decision variables.

Any CSP would be described by this triple $\langle X, D, C \rangle$. A **solution** of a Constraint Satisfaction Problem is an assignment of values to the decision variables which satisfies all the constraints at the same time.

2.2 Constraint Optimization Problem

Previously, we talked about **satisfaction**: since we have constraints, the solution must satisfy all of them simultaneously. Anyway, sometimes we need to deal with **optimization**, that it's the same thing as before but there is an additional term, named **objective function**.

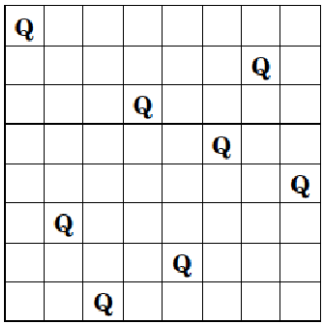
In this case, the formal definition of a **Constraint Optimization Problem** is described also by f, the formalization of the optimization criterion as an objective function. Therefore, our original triple becomes $\langle$**X,D,C,f**$\rangle$, where:

- **X** set of decision variables.
- **D** set of domains of the decision variables.
- **C** set of constraints applied to the decision variables.
- **f** formalization of the optimization criterion. It can be a **minimization objective** (*minimize* f) or a **maximization objective** (*maximize* $-f$).

Let's take a look at the following problems, understanding what decision variables, domains and constraints are really.

N-Queens

e.g. Place n queens in an $n \times n$ board so that no two queens can attack each other.



There are many ways to model this problem, however we try to solve it using the obvious approach. Our goal is to place n queens in a such way there is only one of them for each row, column and diagonal. We begin by defining the decision variables and their domains.

Decision variables and domains.

- Since each row must contain exactly only one queen, we define n decision variables $[X_1, X_2, \ldots, X_n]$; mapping one variable at each row we can satisfy the *no row attack* condition.

- The domain values $\{1, \ldots, n\}$ represent the available columns. An assignment $X_i = j$ indicates that queen in row i is positionated in column j.

To ensure no two queens attack each other, we must address three types of constraints:

1. *Row attacks.*
2. *Column attacks.*
3. *Diagonal attacks.*

Our modelling choice about decision variables allows us to satisfy the row constraint by design. Because each X_i represents a distinct row, we only need to explicitly define constraints for the remaining two.

Constraints.

- **alldifferent**$([X_1, X_2, \ldots, X_n])$.
- for all $i < j$ $|X_i - X_j| \neq |i - j|$.

These constraints prevent column and diagonal attacks, respectively. Specifically, the second constraint ensures that no two queens occupy the same diagonal, which occurs when the distance between their colums is equal to the distance between their rows.

Sudoku

e.g. Sudoku.

	6		1		4		5	
		8	3		5	6		
2								1
8			4		7			6
		6				3		
7			9		1			4
5								2
		7	2		6	9		
	4		5		8		7	

Solving the famous sudoku problem, we can do something similar to the previous example. Even in this problem, we get some decision making processes subject to constraints. As before, we begin by defining the decision variables.

Decision variables.

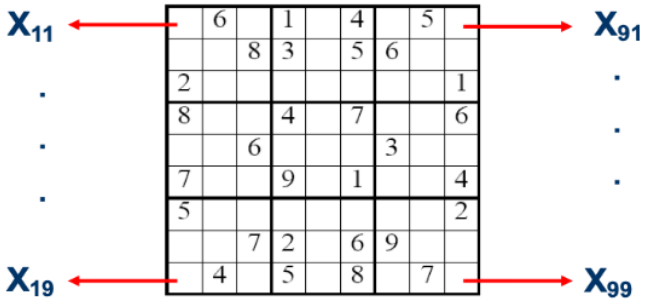
We start modelling the decision variables, which are in this case the **empty cells**: for every cell we have to give a decision. Anyway, focusing only on the empty cells we cannot model properly the given problem; also, the already

assigned cells can somehow influence our decisions.

So, what do we do?
Simply, in our model these variables will be assigned to the current values shown in the image.

Domains.
The domain values are any value in the set $\{1, \ldots, 9\}$.

Summing up, we can create 9×9 different decision variables, where each of them has a domain $\{1, \ldots, 9\}$; except for those variables that are already associated to a value.



Constraints.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, \ldots, X_{91}])$.
- **alldifferent**$([X_{11}, X_{12}, X_{13}, \ldots, X_{19}])$.
- **alldifferent**$([X_{11}, X_{21}, X_{31}, X_{12}, X_{22}, X_{32}, X_{13}, X_{23}, X_{33}])$.
- Initial assignments, already described in the image (*for instance,* $X_{21} = 6$).

The first three constraints ensure different values for any row, column and box of the sudoku table.

Map colouring

e.g. Optimal map colouring.

Our goal is to colour regions such that neighboring regions are assigned different colors, using the minimum number of colors.

Decision variables and domains.
Every decision variable would be a region and any single domain value represents a different color. More formally, X_i with $i \in \{1, \dots, n\}$ are the decision variables, while $\{1, \dots, n\}$ it's the domain.

Since we have n regions, we could use n different colors satisfying the map colouring problem. However, we want to find out the minimum number of colors that we can use.

Constraints.
- $X_i \neq X_j$ for each neighboring region i and j.

We specify inside the constraint the necessary condition for stating when a region is a neighbor of an other one, since not all the regions are consecutive between each other.

Objective variable.
- $f = max(X_i)$
  where:
  - $X_i \rightarrow$ number of colors used by the map colouring
  - $max(\dots) \rightarrow$ function used to detect how many colors were used

Objective.
In order to obtain the minimum number of colors for the map colouring problem we have to minimize the objective function f.

2.3 Task scheduling

Constraint Programming is really powerful when we have to deal within **scheduling problems**. Typically, such problems are described by the following definition.

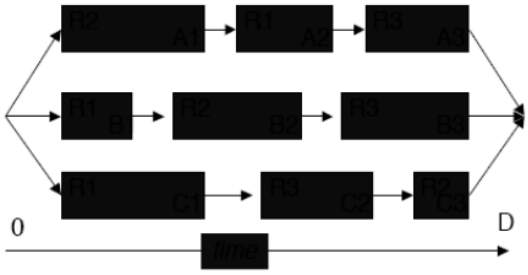
Task scheduling definition

Schedule n tasks on a machine, in time D, by obeying the temporal and precedence constraints:
1. Each task t_i has a specific fixed processing time p_i.
2. Each task t_i can be started after its release date r_i and must be completed before its deadline d_i.
3. Tasks cannot overlap in time.
4. Precedence relations must be respected.

Example

e.g. Task scheduling general problem.

Before we get into the solution, let's take a look at the image below.



Observing the figure, we can state:

- Every black box is a task t_i and its size determines the fixed processing time p_i.
- Every flow does not overlap tasks in the same row.
- Every flow respects the precedence relationships.

We could express decision variables, domains and constraints of any kind of task scheduling problem following a generic approach.

Generic approach.
Since the more interisting part of the optimization problem is the starting time of tasks t_i, the same decision variables can be directly described as the starting time; this is possible since we already know how much time any process requires.

Decision variables and domains.
We call $Start_i$ the decision variables representing the starting time of tasks t_i, where each of them takes a value from $[0 \dots D]$.

The domain $[0 \dots D]$ defines all the time we have available to compute our tasks t_i.

Constraints.

- Respect of release date and deadline.
  - for all $i \in \{1, \dots, n\}$ such that $r_i \leq Start_i \leq d_i - p_i$
    where:
    * $r_i \rightarrow$ release time
    * $d_i \rightarrow$ deadline time
    * $p_i \rightarrow$ processing time
    * $Start_i \rightarrow$ starting time of the task t_i

- No overlap in time.
  - for all $i < j \in \{1, \ldots, n\}$ such that $(Start_i + p_i \leq Start_j) \vee (Start_j + p_j \leq Start_i)$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $p_j \rightarrow$ processing time of the task t_j
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
- Precedence constraints.
  - $Start_i + p_i \leq Start_j$ for each pair of tasks $t_i \mapsto t_j$
    where:
    * $p_i \rightarrow$ processing time of the task t_i
    * $Start_i \rightarrow$ starting time of the task t_i
    * $Start_j \rightarrow$ starting time of the task t_j
    * $t_i \mapsto t_j \rightarrow$ precedence relation between tasks t_i and t_j

Final mark.
Before moving on, let's take a look at the final two constraints.

2nd No overlap in time.
The starting and ending time of any pair of tasks (t_i, t_j) must be either the same time or ordered time, but they are never allowed to overlap each other.

3rd Precedence constraints.
In almost of the cases, the second part of the previous constraint doesn't even exist ($\vee$ $(Start_j + p_j \leq Start_i)$), since in general the output of a task it's used by another task as input, relying on the precedence relationships.

2.4 Constraints

Constraints fall into two broad categories:

1. Most of them rely on declarative relations among objects. They describe an **intensional representation**, since we show the mathematical relationship in a **declarative** way of the constraint (*e.g.* $X > Y$).
2. Any constraint can be expressed by listing all the allowed combinations. This set of constraints follows exactly the mathematical relationship definition: any constraint can be seen as a subset of the cartesian product of all the domains. Generally, these constraints are named **extensional representation** or **table constraints** (*e.g.* $C(X_1, X_2) = \{(0,0), (0,2), (1,3,(2,1))\}$).

Additionally, constraints share the same features, in particular:

- **Order does not matter**.
$$X + Y \leq Z \text{ is the same as } Z \geq X + Y$$
- **Non-directional**.
A constraint between decision variables X and Y can be used to infer domain information on Y given domain information on X and vice versa.
- **Rarely independent**.
Shared variables as communication mechanism between constraints. For instance, using the following relations $C_1 \rightarrow C_2 \rightarrow C_3$, we can exploit C_2 to infer something between C_1 and C_3.

Intensional constraints examples

e.i. There are several constraints regarding intensional representation category.

- **Algebraic expressions**.
  - $X_1 > X_2$
  - $X_1 + X_2 = X_3$
- **Extensional constraints**.
$$(X, Y, Z) \text{ in } \{(1, 3, 5), (2, 4, 6), (3, 5, 4)\}$$
- **Logical relations**.
$$(X < Y) \vee (Y < Z) \rightarrow C$$
- **Global constraints**.
  - $\mathit{alldifferent}([X_1, X_2, X_3])$ instead of:
    $X_1 \neq X_2, X_1 \neq X_3, X_2 \neq X_3$
  - $\mathit{noOverlap}([\mathit{Start}_i, \ldots, \mathit{Start}_n], [p_1, \ldots, p_n])$ instead of:
    for all $i < j \in \{1, \ldots, n\}, (\mathit{Start}_i + p_i \leq \mathit{Start}_j) \vee (\mathit{Start}_j + p_j \leq \mathit{Start}_i)$

  Rule of thumb: if we have a relationship between all the decision variables, we always choose the **global constraint** instead of the hand-made one.
- **Meta-constraints**.
  A meta-constraint is a constraint inside an other constraint.
$$\sum_i (X_i > t_i) \leq 5$$
  where:
  - $X_i \rightarrow$ decision variable
  - $t_i \rightarrow$ parameter

  We are simply counting how many decision variables satisfy the actual

constraint. Roughly speaking, it's created a hidden boolean variable and any time $X_i > t_i$ the variable is set to 1. We merge these boolean variables swearing that the decision variables are at most 5.

2.5 Modelling is critical!

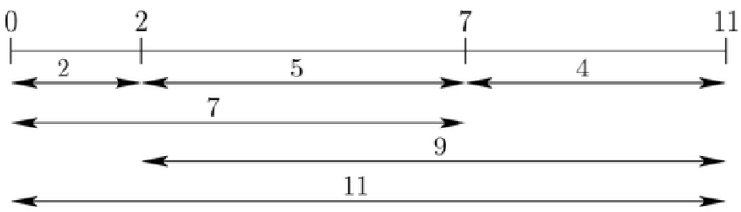
Every choice done can affect our model, like:

- **Choice of decision variables and their domains**.
  The search space is defined as the product of all domains for each decision variable, in particular:
  $$|D(X_1)| \times |D(X_2)| \times \cdots \times |D(X_n)|$$
  Therefore, the search space could be **exponential** in size.
- **Choice of constraints**.
  Selecting the ideal constraints might be the essential key to solve our optimization problem. Propagating constraints the size of the search space can be reduced.

2.6 Golomb ruler

Golomb ruler problem

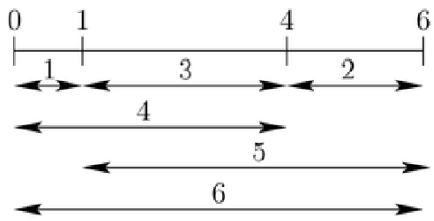
e.g. Starting from an empty ruler we have to place m marks.



At first sight, the problem may appear easy to solve, but instead it's really difficult. Our goal is to place m different marks on a ruler such that the following conditions are satisfied:

1. Distance between each pair of marks is different.
2. The length of the ruler obtained is minimum.

As shown in the image above, we placed *four marks* over the ruler, achieving a total length equal to 11, even though this is not the optimal solution.



With this second approach, we reach a solution up to 6 always placing *four different marks*.

Decision variables and domains.
It's quite easy to determine the given marks as m different decision variables, but what makes the Golomb ruler problem so difficult is the choice of their domains, in particular:

- $[X_1, X_2, \ldots, X_m]$ are our decision variables.
- $\{0, 1, \ldots, 2^{m-1}\}$ is the domain of each decision variable. Often the upper limit coincides within 2^{m-1}, since also in the worst case we are able to place m marks using the powers of 2, guaranteeing at least one solution.

Constraints.

- for all $i_1 < j_1, i_2 < j_2, i_1 \neq i_2$ or $j_1 \neq j_2$ $|X_{i1} - X_{j1}| \neq |X_{i2} - X_{j2}|$

Making sure that all pairs of distances have a different difference.

Objective.
Since we don't know which decision variable will have the highest value, we have to minimize the max value (*the max must be minimum as much as possible*).

$$minimize(max([X_1, X_2, \ldots, X_m]))$$

Final mark.
It's not a good model, since we have absolute values in both sides of the constraint, making hard the propagation. It would be nice to delete these absolute values to get a smoother model. We need to remodel.

Sometimes we introduce **additional variables** in our model, even though we said previously more variables we have more large would be the search space. However, adding variables in order to obtain a bigger search space helps us to express constraints much better than before and, also, spread their propagations.

Better model

e.g. Starting from an empty ruler we have to place m marks.

Auxiliary variables.

In the Golomb ruler problem we can introduce a new additional variable D_{ij} that represents the distance between mark i and mark j.

Constraints.
The new constraints should be as follows:
- for all $i < j,\ D_{ij} = |X_i - X_j|$
- *alldifferent*$([D_{12}, D_{13}, \ldots, D_{(m-1)m}])$

Thanks to the additional variable D_{ij} we can directly express a global constraint.
- for all $i < j,\ D_{ij} = |X_i - X_j|$
- *alldifferent*$([X_1, X_2, \ldots, X_m])$
- *alldifferent*$([D_{12}, D_{13}, \ldots, D_{(m-1)m}])$

The second constraint introduced is named **implied constraint**, as the same name suggests, this is a constraint **logically redundant** but **computationally significant**. *alldifferent*$([X_1, X_2, \ldots, X_m])$ is not a new constraint, since different distances (*the third constraint*) already implies that also decision variables $X_1, \ldots, X_m$ must be different.

2.7 Symmetry in CSPs

The model designed could have **symmetry**: a state leading to a solution or a failure that have many symmetrically equivalent states. Symmetry is divided into:

- **Variable symmetry**.
  A **permutation** π of the **variable indeces** such that for each feasible or unfeasible assignment, we can rearrange the variables according to π and obtain another feasible or unfeasible assignment.

  Variable symmetry in Golomb ruler problem

  e.g. Permuting variable assignments.

  $$\langle X_1 = 0, X_2 = 1, X_3 = 4, X_4 = 6 \rangle$$

  Starting from the above solution, we can define different ways to assign values or marks over the ruler, obtaining always the same conclusion. For any solution or failure composed by m entities, we can derive $m!$ permutations. For instance, looking at the previous example we get $4 \times 3 \times 2 = 24$ same conclusions.

- **Value symmetry**.

A **permutation** π of the **values** such that for each feasible or unfeasible assignment, we can rearrange the values according to π and obtain another feasible or unfeasible assignment.

Value symmetry in Golomb ruler problem

e.g. Values can be permuted as follows:

$$\langle 0 \to 0, 1 \to 2, 2 \to 1, 3 \to 3, 4 \to 5, 5 \to 4, 6 \to 6 \rangle$$

Even for value permutations we get the same factorial complexity, $m!$ where m is the number of values assigned.

2.8 Symmetry breaking constraints

We need a way to reduce the set of solutions and consequently the huge search space from the symmetries. A general approach is to impose an **ordering** between the decision variables to avoid the permutations, in particular:

$$X_1 \leq X_2 \leq \cdots \leq X_n \text{ when } [X_1, X_2, \ldots, X_n] \text{ are all symmetric.}$$

Notice, at least one solution from each set of symmetrically equivalent solutions must remain.

Improved model

Symmetry breaking constraints.
- $X_1 < X_2 < \cdots < X_m$
- $X_1 = 0$
- $D_{12} < D_{(m-1)m}$

Notice: symmetry breaking constraints and implied constraints are not the same thing. An implied constraint is a logical implication that does not change the solution at all, while a symmetry breaking constraint is not a logical implication, having it inside our model could change the state of the solution.

New objective.
- $minimize(X_m)$

At first sight, any global constraint seems to be a good constraint for our solver, like *alldifferent*(...) constraint. However, after we introduced these symmetry breaking

constraints, the *alldifferent*(...) constraint is not useful anymore, it becomes **semantically** and **computationally redundant**. Symmetry breaking constraints enable **constraint semplification**.

Simplified improved model

Symmetry breaking constraints.

- $X_1 < X_2 < \cdots < X_m$
  alldifferent$([X_1, X_2, \ldots, X_m])$ constraint becomes redundant. In the process of remodelling, delete *alldifferent*$([X_1, X_2, \ldots, X_m])$ constraint can be a good idea to avoid redundancy. In addition, also the first constraint introduced in the previous section (*for all* $i < j,\ D_{ij} = |X_i - X_j|$) can be simplified as follows:
  - for all $i < j,\ D_{ij} = X_i - X_j$

  We don't need anymore absolute values, exploiting the decision variables order imposed by symmetry breaking constraints.

2.9 Dual viewpoint

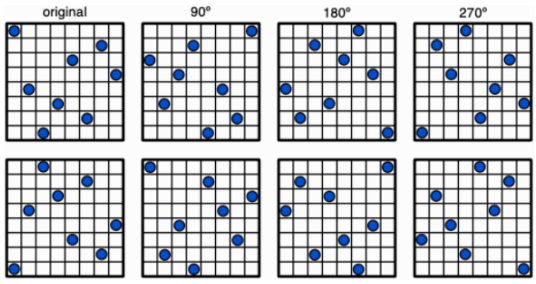
Modelling phase is critical. Starting from a new problem and observing it by different **perspectives** we can design several models. Generally, every model exhibits a different representation of the original problem, in particular any of them have:

1. Different decision variables.
2. Different domains.
3. Different constraints.
4. Different size of the search space.

Which model is the best one?

Symmetries of n-queens problem

e.g. Solution of n-queens problem.



Just rotating and flipping the board we end up with *seven* additional symmetric solutions of the top-left original one. Additionally, we cannot impose an ordering constraint as we did before for the ruler problem. We need a new **viewpoint** to avoid these *seven* symmetric feasible solutions.

Decision variables and domains.
We represent the $n \times n$ board as a boolean board and every decision variable is a boolean variable like $B_{ij} \in \{0, 1\}$.

Attacking constraints.
- $\sum B_{ij} = 1$ on all rows and columns.
- $\sum B_{ij} \leq 1$ on all the diagonals.

Now we are able to convert geometric symmetry to boolean symmetry.

Symmetry breaking constraints.
We determine the following list of steps to avoid symmetric solutions.

1. Flat the 2-D matrix to a single sequence of variables. Simply, we take each row of the board and we append it at the end of the sequence.
2. Every symmetric configuration corresponds to a **variable permutation** of the original solution, which is easy to define.
3. We then impose an order between a solution and all the solutions obtained by the *seven* permutations:

$$lex \leq (B, \pi(B)) \text{ for all } \pi$$

where:
- $B \rightarrow$ flatten version of the 2-D matrix
- $\pi(B) \rightarrow$ symmetries of the original solution

Chapter 3

Local consistency, constraint propagation and global constraints

After the second chapter, we are able to design a model composed by decision variables and constraints. In addition, we stated that the choices done during the modelling phase could affect the size of the search space and the performance of constraint propagation algorithm. Anyway, what is a **constraint propagation algorithm**?

Constraint propagation

Constraint propagation is a process used to examine every constraint to **remove inconsistent values** from the domains of the **future decision variables**.

3.1 Local consistency

Before, we mentioned the term **consistency**, but, actually, what does consistency really mean?

Local consistency

Local consistency is a form of logical inference which detects **inconsistent partial assignments**.

Before moving on, let's take a look at the previous definition, in particular we said about assigments:

- **Partial**, since we assign gradually values to the decision variables.
- **Inconsistent**, assignments that are not part of the final solution should be deleted.

In addition, we are dealing with a certain type of consistency, named **local consistency**: we examine only one constraint at time, we never take in account more than one constraint. Typically solvers use local consistencies domain-based, they detect inconsistent partial assignments of the form $X_i = j$, removing j from the domain of X_i, so $j \in D(X_i)$, if this is the case.

3.2 Generalised Arc Consistency

Generalised Arc Consistency, briefly **GAC**, is a general form of arc-consistency, that takes in account more than two decision variables at time. Let's see a formal example about it.

Formal example

Any constraint C defined on k decision variables, $C(X_1, \ldots, X_k)$, gives us the following set of allowed combinations of values:

$$C \subseteq D(X_1) \times \cdots \times D(X_k)$$

Remember, a constraint is a mathematical relationship over a subset of the cartesian product of all decision variables domains.

Each allowed tuple $(d_1, \ldots, d_k) \in C$ is a **support** for C.

Therefore, we say that the general constraint $C(X_1, \ldots, X_k)$ is **GAC** if and only if:

$$\forall X_i \in \{X_1, \ldots, X_k\} \text{ and } \forall v \in D(X_i) \text{ for } i \in \{1, \ldots, k\}, v \in C$$

A Constraint Satisfaction Problem is **GAC** if and only if all its constraints are **GAC**.

Arc-consistency is a more specific case about Generalised Arc Consistency, where the total number of decision variables taken in account is equal to $k = 2$, so the set of decision variables is $\{X_1, X_2\}$.

Example

e.g. Arc-consistency.

$$D(X_1) = \{1, 2, 3\}, D(X_2) = \{2, 3, 4\}, C : X_1 = X_2$$

Is constraint C arc-consistent?

Some values of the decision variables domains do not have a **support**, in particular:

- $1 \in D(X_1)$ and $4 \in D(X_2)$ do not have a support respectively in $D(X_2)$ and $D(X_1)$.
- $X_1 = 1$ and $X_2 = 4$ are inconsistent partial assignments, so they should be deleted from their domains.

Even though there are some values able to satisfy the constraint, C is not arc-consistent since every value does not have a **support**.

Example

e.g. Generalised Arc Consistency.

$$D(X_1) = \{1, 2, 3\}, D(X_2) = \{1, 2\}, D(X_3) = \{1, 2\}, C : \mathit{alldifferent}([X_1, X_2, X_3])$$

Is constraint C GAC?

It's quite easy to state that this constraint is not in GAC, but anyway more formally we have to say:

- $1 \in D(X_1)$ and $2 \in D(X_1)$ do not have a support.
- $X_1 = 1$ and $X_1 = 2$ are inconsistent partial assignments.

If we are able to detect where the inconsistencies are, thanks to propagation, we can remove these values from the decision variables domains, achieving our main goal.

3.3 Constraint propagation

Applying constraint propagation, we can achieve local consistency of any constraint C (*examing only one constraint at time*). Therefore, **local consistency** can be seen as our target, while **constraint propagation** is the process to achieve this target.

The basic behavior of a constraint propagation algorithm consists in removing the

inconsistent values from the decision variables domains involved in C.

Example

e.g. Arc-consistency.

$$D(X_1) = \{1, 2, 3\}, D(X_2) = \{2, 3, 4\}, C : X_1 = X_2$$

Is constraint C arc-consistent?

As before, we know that:
- $1 \in D(X_1)$ and $4 \in D(X_2)$ do not have a support respectively in $D(X_2)$ and $D(X_1)$.
- $X_1 = 1$ and $X_2 = 4$ are inconsistent partial assignments, so they should be deleted from their domains.

Deleting these values described above, we can reach arc-consistency for the constraint C:

$$D(X_1) = \{\bar{1}, 2, 3\}, D(X_2) = \{2, 3, \bar{4}\}$$

The new domains for X_1 and X_2 will be both $D(X_1) = D(X_2) = \{2, 3\}$. Now constraint C is **arc-consistent**.

Example

e.g. Generalised Arc Consistency.

$$D(X_1) = \{1, 2, 3\}, D(X_2) = \{1, 2\}, D(X_3) = \{1, 2\}, C : \mathit{alldifferent}([X_1, X_2, X_3])$$

Is constraint C GAC?

As before, we state that:
- $1 \in D(X_1)$ and $2 \in D(X_1)$ do not have a support.
- $X_1 = 1$ and $X_1 = 2$ are inconsistent partial assignments.

We can remove these values from the domain of X_1 in order to achieve consistency, in particular:

$$D(X_1) = \{\bar{1}, \bar{2}, 3\}$$

The new domain for the decision variable X_1 will be $D(X_1) = \{3\}$. Now constraint C is **GAC**.

Furthermore, removing **incompatible values** from the domains of the decision variables is not only a way to achieve consistency but also a way to shrink the search space, improving the performance of our solver.

It doesn't exist only GAC or arc-consistency as local consistency: both of them has a very efficient propagation algorithm, but sometimes it might be difficult to achieve this level of consistency or its computational complexity might be too huge, requiring a lot of time. In such cases, we have to focus on lower levels of consistency, detecting less incosistent values.

3.4 Bounds Consistency

Bounds Consistency, briefly **BC**, is a lower level of consistency than GAC or arc-consistency. Usually, it's used when all the domain values are ordered, since it relaxes the domain of any decision variable X_i from $D(X_i)$ to $[min(X_i), \ldots, max(X_i)]$. So by relaxing we take in account the whole interval, limited by the minimum and maximum values coming from the original domain.

Example

e.g. Relaxing a decision variable domain.

$$D(X_i) = \{1, 3, 5\} \rightarrow [1 \ldots 5]$$

In BC, we don't talk anymore about **support** but instead about **bound support**, since values don't come anymore from a precise domain but from a more large range.

Formal example

Any constraint C defined on k decision variables, $C(X_1, \ldots, X_k)$, gives us the following set of allowed combinations of values:

$$C \subseteq D(X_1) \times \cdots \times D(X_k)$$

Each allowed tuple $(d_1, \ldots, d_k) \in C$ where $d_i \in [min(X_i), \ldots, max(X_i)]$ is a **bound support** for C.

Therefore, we say that the general constraint $C(X_1, \ldots, X_k)$ is BC if and only if:

$$\forall X_i \in \{X_1, \ldots, X_k\}, min(X_i) \text{ and } max(X_i) \text{ belong to a bound support}$$

Previously, the GAC requirement was totally different, any value $v \in D(X_i)$ must belong to a whatever **support**, but now we care only about the **minimum** and the **maximum** values. By BC we make the search fastest than before, because we need to check only two values and not each of them.

However, everything comes with advantages and disadvantages, in particular:

- **Disadvantage**.
  1. BC might not detect all GAC inconsistencies. BC checks only the minimum and maximum values, between them might be different undected inconsistencies.
- **Advantages**.
  1. Might be easier to look for a support in a range than in a domain, in particular when domains are very large.
  2. Achieving BC is enough to achieve also GAC when we are dealing with **monotonic constraints**.
  3. Achieving BC is often cheaper than achieving GAC.

Example

e.g. Differences between GAC and BC regarding *alldifferent*$(\dots)$ constraint.

$$D(X_1) = D(X_2) = D(X_3) = \{1, 3\}, C : \textit{alldifferent}([X_1, X_2, X_3])$$

It's quite easy to see that the constraint C is unsatisfiable, the key distinction lies in when GAC and BC actually recognize this impossibility.

GAC would be perfectly aware that there are three different decision variables and only two available values in the domain and it will just fail. BC won't see that, since it uses a range instead of the original domain, assuming also 2 as available value for the future assignments. Anyway, also BC will discover that this constraint is unfeasible, but this will occur later than by GAC.

Technically, things go as follows:
- BC(C).
  - All $min(X_i)$ and $max(X_i)$ belong to a bound support.
  - No domain reduction with BC propagation, since the tuple d_i belongs to the bound support of C.
- GAC(C).
  - None of $min(X_i)$ and $max(X_i)$ belongs to a support.
  - C fails with GAC propagation.

Example

e.g. Differences between GAC and BC regarding **monotonic constraint**.

$$D(X_1) = [2 \dots 6], D(X_2) = [1 \dots 5], C : X_1 \leq X_2$$

In this example, GAC and BC will give the same reasoning, so we can summerize exactly the same behavior as follows:

- All values $D(X_1)$ must be less or equal to $max(X_2)$.
- All values of $D(X_2)$ must be greater or equal to $min(X_1)$.
- It's enough to adjust the maximum value of X_1 and minimum value of X_2 to achieve consistency.

After that, the final domains will be both $D(X_1) = D(X_2) = [2 \dots 5]$.

3.5 Propagation algorithm

When we are solving a complex Constraint Satisfaction Problem, constraints are rarely isolated. Because different constraints often share the same decision variables, a change in one variable's domain can trigger a chain reaction. As we already saw, any constraint has its own propagation algorithm and since they could have *shared resources* a given propagation algorithm can wake up an already propagated constraint to be repropagated again.

At the end, propagation reaches a **fixed point** and all constraints reach a level of consistency. The whole process described in few words before is called **constraint propagation**.

Example

e.g. Constraint propagation.

$$D(X_1) = D(X_2) = D(X_3) = \{1, 2, 3\},$$
$$C_1 : \mathit{alldifferent}([X_1, X_2, X_3]), C_2 : X_2 < 3, C_3 : X_3 < 3$$

Assuming the order of propagation is $\langle C_1, C_2, C_3 \rangle$ and we would like to achieve GAC consistency, the propagation process goes as follows:

1st Propagation of C_1.
- Nothing changes, C_1 is already in GAC.

2nd Propagation of C_2.
- $X_2 = 3$ is an inconsistent assignment, so we need to remove it from its domain $D(X_2) = \{1, 2, \bar{3}\}$.
- Removing 3 from the decision variable domain, $D(X_2) = \{1, 2\}$, C_2 is now in GAC.

3rd Propagation of C_3.

- $X_3 = 3$ is an inconsistent assignment, so we need to remove it from its domain $D(X_3) = \{1, 2, \bar{3}\}$.
- Removing 3 from the decision variable domain, $D(X_3) = \{1, 2\}$, C_3 is now in GAC.

4th Propagation of C_1.
C_1 is not in GAC anymore, since the supports of $\{1, 2\}$ values have been removed from the propagation of C_2 and C_3.

- $X_1 = 1$ and $X_1 = 2$ are inconsistent assignments, so we need to remove them from its domain $D(X_1) = \{1, \bar{2}, \bar{3}\}$.
- Removing 2 and 3 from the decision variable domain, $D(X_1) = \{3\}$, C_1 is again in GAC.

As we can state from the example above, sometimes it may not be enough to remove inconsistent values from domains once, some propagation algorithm can be re-triggered, starting again the whole propagation process. Anyway, a propagation algorithm **must wake up when necessary**, otherwise we may not achieve the desired local consistency.

About the local consistencies seen so far, the following events may trigger their propagation algorithm:

- GAC, when the domain of a variable changes.
- BC, when the domain bounds of a variable changes.
- Both of them, when a value is assigned to a decision variable.

Example

e.g. Constraint propagation.

$$D(X_1) = D(X_2) = D(X_3) = \{1, 2, 3\},$$
$$C_1 : \mathit{alldifferent}([X_1, X_2, X_3]), C_2 : X_2 \neq 2, C_3 : X_3 \neq 2$$

Assuming the order of propagation is $\langle C_1, C_2, C_3 \rangle$, where C_1 maintains BC and AC the others. The propagation process goes as follows:

1st Propagation of C_1.

- Nothing changes, C_1 is already in BC.

2nd Propagation of C_2.

- $X_2 = 2$ is an inconsistent assignment, so we need to remove it from its domain $D(X_2) = \{1, \bar{2}, 3\}$.
- Removing 2 from the decision variable domain, $D(X_2) = \{1, 3\}$, C_2 is now in AC.

3rd Propagation of C_3.

- $X_3 = 2$ is an inconsistent assignment, so we need to remove it from its domain $D(X_3) = \{1, \bar{2}, 3\}$.
- Removing 2 from the decision variable domain, $D(X_3) = \{1, 3\}$, C_3 is now in AC.

4th Propagation of C_1.

There's no need to re-trigger C_1 constraint because it only maintains BC consistency. Since the changes done were limited to intermediate values and did not affect the domain bounds, the constraint remains consistent and it's not necessary to wake up it again.

How much expensive is a propagation algorithm? Assuming a Constraint Satisfaction Problem where all the decision variables have the same domain size, $|D(X_i)| = d, \forall i \in \{1, \dots, k\}$, achieving a local consistency by AC propagation for a whatever constraint $C(X_1, X_2)$ takes $O(d^2)$ time. It's not a good complexity, since in almost of the cases we have to check the domain of every variable. However, we can do better than this, using some tricks.

Example

e.g. AC propagation tricks.

$$C : X_1 = X_2$$

The naive way would be: check if for every value of X_1 there is a support value in the domain of X_2. The same constraint can be expressed as follows:

$$D(X_1) = D(X_2) = D(X_1) \cap D(X_2)$$

Therefore, the propagation algorithm should be activated when the **intersection** of the two domains changes.

$$C : X_1 \neq X_2$$

When the domain of a decision variable coincides with a single value, $D(X_i) = \{v\}$, we remove it from the domain of the second variable, $D(X_j) = \{\dots, \bar{v}, \dots\}$. Obviously, we don't remove it when both variables have multiple domains, because they can acquire different values.

$$C : X_1 \leq X_2$$

Without checking all the possible combinations of values, we can express the same relation in the following way:

$$max(X_1) \leq max(X_2), min(X_1) \leq min(X_2)$$

If we can satisfy these conditions, we can state that our constraint is arc-consistent.

3.6 Specialized propagation

Specialized propagation is a propagational algorithm based on the **semantic** of the constraint. So, we don't put its propagation in a naive way but we do it in the fastest way exploiting the semantic of that specific constraint.

Such approach can be potentially much more efficient than a general approach, but it's not always easy to develop.

As we already know, **global constraints** are very useful for the modelling phase, but at the same time they embed specialized propagation algorithms making the propagational process much more effective than the other ways to express the same complex constraints.

We can see global constraints benefits in two perspectives:

- **Modelling benefits**.
  - Reduce the gap between the problem description and the formal description (*the model*).
  - May allow the expression of constraints that are otherwise not possible to state using primitive constraints.
- **Solving benefits**.
  - Efficient propagation.
  - Strong inference in propagation.

Therefore, as rule of thumb, any time we have some complex relationships to express, we should check the existing global constraints and see if we can exploit any of them.

3.7 Some global constraints

We are gonna to examine the following global constraints:

1. Counting
2. Sequencing
3. Scheduling
4. Ordering

1st **Counting constraint**.
It restricts the number of variables satisfying a condition or the number of times values are taken. An example, that we already know it, is the *alldifferent(...)* constraint.

alldifferent(...)

$$alldifferent([X_1, \dots, X_k]) \text{ holds if and only if}$$
$$X_i \neq X_j \text{ for } i < j \in \{1, \dots, k\}$$

In this global constraint, there is a hidden counting because we are observing how many times a value is assigned to a decision variable, that it can be at most one single time.

Another counting constraint is the **global cardinality constraint**, briefly **gcc**. Gcc constrains the number of times each value is taken by the variables, in particular:

gcc$(\dots)$

$$gcc([X_1, \dots, X_k], [v_1, \dots, v_m], [O_1, \dots, O_m]) \text{ holds if and only if}$$
$$\forall j \in \{1, \dots, m\}\ O_j = |\{i | X_i = v_j, 1 \leq j \leq k\}|$$

where:
- $[X_1, \dots, X_k] \rightarrow$ decision variables
- $[v_1, \dots, v_m] \rightarrow$ allowed values to assign
- $[O_1, \dots, O_m] \rightarrow$ counters about how many times a value v_j has been assigned

alldifferent$(\dots)$ constraint can be seen as a specific case of **gcc**, where counters O_j must be equal to 1.

A third counting constraint is the *among*$(\dots)$ constraint. It constrains the number of variables taking certain values, in particular:

among$(\dots)$

$$among([X_1, \dots, X_k], s, l, u) \text{ holds if and only if}$$
$$l \leq |\{i | X_i \in s, 1 \leq k \leq k\}| \leq u$$

where:
- $s \rightarrow$ set of allowed values

- $l \rightarrow$ lower bound
- $u \rightarrow$ upper bound

Example

$$among([1, 5, 3, 2, 5, 4], \{1, 2, 3, 4\}, 3, 4)$$

It states: only between 3 and/or 4 decision variables can take values in $\{1, 2, 3, 4\}$.

2st **Sequencing constraint**.
It ensures a sequence of variables obey certain patterns. As before, we consider some examples about this family of constraint.

The *sequence*(...) constraint limits the number of values taken from a given set in any subsequence of q variables, in particular:

sequence(...)

$$sequence(l, u, q, [X_1, \ldots, X_k], s) \text{ holds if and only if}$$
$$among([X_i, X_{i+1}, \ldots, X_{i+q-1}], s, l, u) \text{ for } 1 \leq i \leq k - q + 1$$

where:
- $l \rightarrow$ lower bound
- $u \rightarrow$ upper bound
- $q \rightarrow$ number of decision variables inside the subsequence
- $s \rightarrow$ set of allowed values

Example

$$sequence(1, 2, 3, [1, 0, 2, 0, 3], \{0, 1\})$$

We are looking for a subsequence of three variables ($q = 3$) where at most one and/or two ($l = 1, u = 2$) of them can have values in $\{0, 1\}$.

3rd **Scheduling constraint**.
It helps to schedule tasks with respective release times, duration, and deadlines, using limited resources in a time interval.

Sometimes we would like to design a scheduling problem, using a particular constraint named *disjunctive*(...). Our decision variables will be the starting time of

any task, combining them such that they do not overlap in time.

disjunctive$(\dots)$

Given tasks $t_1, \dots, t_k$ each associated with a start time S_i and duration D_i, the *disjunctive*$(\dots)$ constraint is defined as follows:

$$\textit{disjunctive}([S_1, \dots, S_k], [D_1, \dots, D_k]) \text{ holds if and only if}$$
$$\forall i < j \ (S_i + D_i \leq S_j) \vee (S_j + D_j \leq S_i)$$

This constraint is very useful when a resource can execute at most one task at time: it ensures that there's only one task running on it.

Sometimes we have the other way round: resources that can be used by more than one task. Tasks can overlap in time, but **cumulative resources** have a specific capacity, so we can't parallelize them in a infinite way.

cumulative$(\dots)$

Given tasks $t_1, \dots, t_k$ each associated with a start time S_i, duration D_i, resource requirement R_i and a resource with a capacity C, the *cumulative*$(\dots)$ constraint is defined as follows:

$$\textit{cumulative}([S_1, \dots, S_k], [D_1, \dots, D_k], [R_1, \dots, R_k], C) \text{ holds if and only if}$$
$$\sum_{i | S_i \leq u \leq S_i + D_i} R_i \leq C \ \forall u \in D$$

4th **Ordering constraint**.
It enforces an ordering between the variables or the values. An example is the **lexicographic ordering constraint**: it requires a sequence of decision variables to be lexicographically less than or equal to another sequence of variables.

lex $\leq (\dots)$

$$\textit{lex} \leq ([X_1, X_2, \dots, X_k], [Y_1, Y_2, \dots, Y_k]) \text{ holds if and only if}$$
$$X_1 \leq Y_1 \wedge$$
$$(X_1 = Y_1 \rightarrow X_2 \leq Y_2) \ \wedge \dots$$
$$(X_1 = Y_1 \wedge X_2 = Y_2 \wedge \dots \wedge X_{k-1} = Y_{k-1} \rightarrow X_k \leq Y_k)$$

This constraint is very useful for breaking solution symmetries.

3.8 Specialized propagation for global constraints

There are two main approaches to develop specialized propagation algorithms for global constraints:

1st **Constraint decomposition**.
The main idea about constraint decomposition is to divide a **global constraint** into smaller and simpler constraints, each of which has a known propagation algorithm. So basically we express a global constraint as a conjuction of smaller constraints propagation algorithms.

Example

e.g. Let's take a look again at the *among*(...) counting constraint.

$$\textit{among}([X_1, \ldots, X_k], s, N) \text{ holds if and only if}$$
$$N = |\{i | X_i \in s, 1 \leq i \leq k\}|$$

Assuming that our solver does not have the *among*(...) constraint resolution, it is just a decomposition as a conjuction of logical constraints and a sum constraint:

1. B_i with $D(B_i) = \{0, 1\}$ for $1 \leq i \leq k$.
2. $C_i : B_i = 1 \leftrightarrow X_i \in s$ for $1 \leq i \leq k$.
3. $C_{k+1} : \sum_i B_i = N$.

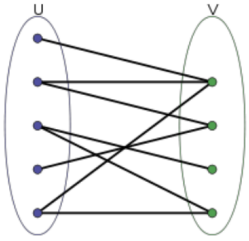
Arc-consistency for C_i and bounds consistency for C_{k+1} ensures **GAC** on *among*(...) constraint.

Almost of the times, this won't be the case: the propagation done might not be complete by decomposition.
2nd **Dedicated propagation algorithms**.
Dedicated algorithms provide effective and efficient propagation. Often, dedicated algorithms can see things better than decomposition, detecting more inconsistent values and maintaining always a polynomial time resolution.

The first GAC propagating algorithm was introduced by Jean-Charles Régin. It was the first example of global constraint computation without the usage of decomposition mechanism. It establishes a relation between the solutions of the constraint and the properties of a graph, providing a **maximal matching** in a **bipartite graph**. Let's see more formally their definitions.

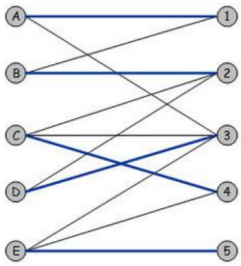
Bipartite graph

A bipartite graph is a graph whose vertices are divided into two disjoint sets U and V such that every edge connects a vertex in U to one in V.



Matching

A matching in a graph is a subset of its edges such that no two edges have a node in common.



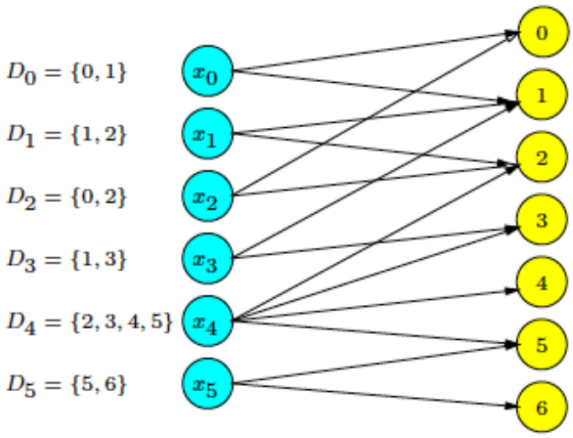
Maximal matching is the largest possible matching. In a bipartite graph, maximal matching covers one set of nodes.

Therefore, combining together decision variables and their values we are able to determine if an assignment of those values is a solution or not.

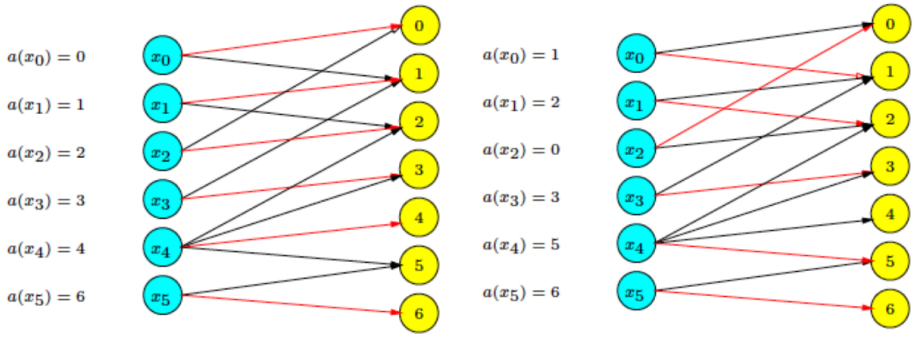
Solution

Given a **bipartite graph** G constructed between the **variables** $[X_1, \ldots, X_k]$ and their possible **values**, an **assignment** of those values is a solution if and only if it corresponds to a **maximal matching** in G (*every couple of edges does not have a node in common*).

Generally, a bipartite graph is composed as in the image below.



The left side represents the set of all the decision variables with their respective domains, while in the right side we have all the different values. Every edge drawn in the previous representation defines a match between a decision variable and a value of its domain. A maximal matching is verified when all the decision variables have a single no-shared match: a variable is linked to a single value.



Chapter 4

Search in CP

After the second and third chapter, we know how to design a model and we understood how constraints propagate. However, when actually these constraints propagate and how do they interleave with the search?

4.1 Constraint solver

Typically, the search is done by a **Backtracking Tree Search (BTS)** algorithm that enumerates all possible variable-value combinations to find a solution. The search tree is built gradually: everytime we need to assign a value to a decision variable a new node representing that variable is created and all the branches that come out of it define the allowed decisions.

4.2 Branching decisions

Let's talk about **branching decisions**. We have several types of branching decisions, described as follows:

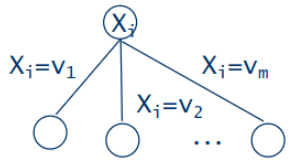
1. **Unary branching**.
   Usually a branching decision consists of posting an **unary constraint** to a single decision variable.
2. **Enumeration branching**.
   By enumeration branching we label a decision variable with a specific value. At the same time enumeration branching is divided into:
   - **d-way branching**.
     We create a separate branch for every possible assignment coming from

the domain of the decision variable. More formally:

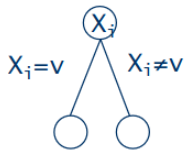
$$\text{one branch is generated by } X_i = v_j \ \forall v_j \in D(X_i)$$

We do not instantiate all these branches at the same time. First of all, we start from the **left-most branch** ($X_i = v_1$) going down in the search, only when we are backtracking we would create the second branch ($X_i = v_2$) and so on.

Note: the letter **d** in **d-way branching** stands for the cardinality of variable's domain.



- **2-way branching**.
  In this case we define only two branches. The left-most branch is an assignment of whatever value coming from the domain and the failing counter-part is the no-assignment of this value to the decision variable.

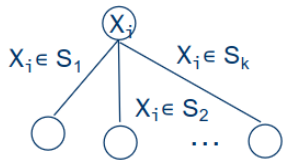


  If we encounter the failing part, the new node will be always the same decision variable but taking a different value from its domain.

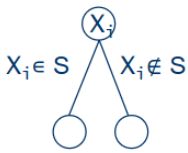
The main difference between these approaches is that in **d-way branching** we have one node per variable, while in **2-way branching** we could have potentially a number of nodes equal to the cardinality of the domain ($|d|$).

Exists also another way to do branching decisions without any assignment, based on restrictions of the domain:

3. **Domain partioning**.
   Simply, we define **partitions** of the domain of the decision variable and we state if the same decision variable can belong or not to one of these partitions. Even in this case, domain partioning is divided into:
   - **k-way branching**.
     We create a specific number of partitions and we define among the branches if any given variable X_i could belong or not to a partion S_j, where $S_j \in D(X_i)$.



- **2-way branching**.
  It follows the same behavior as before: we start from the left-most branch assuming that the decision variable belongs to the partition S ($X_i \in S$), if not we encounter the failing part and we determine a new 2-way branch with a different partition.



Most of times, solvers implement the **2-way branching** approach. By the way, the thing is: how do we know which decision variable X_i should we pick? Or: which partition should we prefer?

The answers to the previous questions are given by the **search heuristics**.

4.3 BTS without propagation

Let's assume a BTS algorithm without propagation at all. In this case, to find out the constraints satisfaction we should try all the possible combinations and test the assignments done. Basically we perform a **Depth-first search strategy** and whenever all the variables of a constraint are instantiated, we check the validity of the constraint.

It may seem quite stupid: first of all we do all the assignments and then we check if the constraint is satisfied or not. However, by this approach we can reach a final solution or prove the unsatisfiability of the problem.

Instead, with propagation **before** the assignments we attempt to remove obvious useless values, or in other words all those values that do not have a **support**.

Example

e.g. BTS example without propagation.

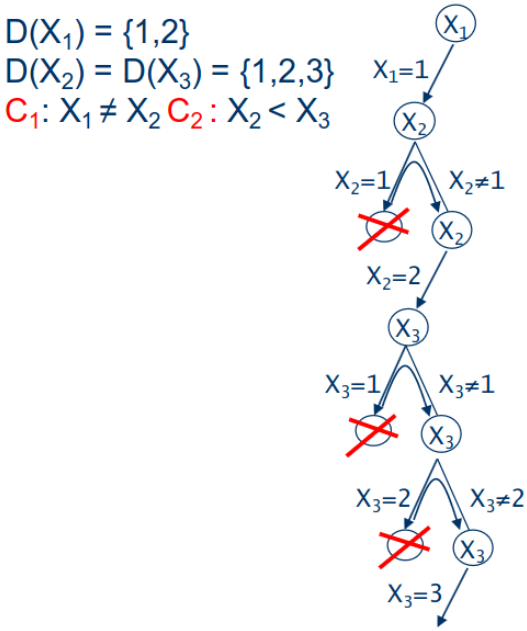
Starting from the below variables, domains and constraints:

$$D(X_1) = \{1, 2\}, D(X_2) = D(X_3) = \{1, 2, 3\}$$
$$C_1 : X_1 \neq X_2, C_2 : X_2 < X_3$$

we can reach the final solution:

$$X_1 = 1, X_2 = 2, X_3 = 3$$

stating the following search tree.



This is a very stupid example: we are assigning values to variables without checking before the requirements of the constraints.

4.4 Constraint solver (BTS + propagation)

Naturally, a constraint solver uses a BTS algorithm within propagation. At each search decision, so at every assignment, it propagates the whole set of constraints to remove inconsistent values from the domains of the future variables.

The main advantages of a mechanism like this are:

1. **Shrink** the domains of the future variables.
2. **Propagate** all the constraints before search, and only when necessary ones

during the search.

However, both approaches, respectively with and without propagation, have the same complexity, equal to:

$$O(d^n)$$

where:

- $d \rightarrow$ domain size
- $n \rightarrow$ number of decision variables

For sure, using the propagation of constraints we can reduce the complexity of BTS algorithm, even though it remains exponential in size.

Example

e.g. Same BTS example as before but with propagation.

Before the search, or the assignments, we start from the propagation, observing which values have a support in the other future variables domains.

$$C_2 : D(X_2) = \{1, 2, \bar{3}\}, D(X_3) = \{\bar{1}, 2, 3\}$$

After the initial propagation, we can already state that values 3 from $D(X_2)$ and 1 from $D(X_3)$ do not have a support. Therefore, we can erase them before the search phase.

Why don't we have also constraint C_1?
Because C_1 is already in AC, so we don't need to propagate. However, after first assignment ($X_1 = 1$) something changes. Since we assign a value to the decision variable X_1, we have to propagate the first constraint and the second one since they share a specific decision variable X_2.

$$C_1 : D(X_2) = \{\bar{1}, 2\}, C_2 : D(X_3) = \{\bar{2}, 3\}$$

4.5 Search heuristics

So far, we understood that the search process is based on three main features: search tree, branching decisions and constraint propagation. Now we move to the topic of search heuristics.

At anytime during the search we have to handle a variable, creating a node representing that variable and the branching decisions coming out of it. So, how do we choose the decision variable and its values?

We divide the **search heuristics** in:

1. **Problem specific heuristics**.
   Looking at the current problem we could identify something and apply it in our heuristic to find a solution.
2. **Generic heuristics**.
   Type of heuristics that can be applied to any problem. Additionally, generic heuristics are divided into:
   - **Static heuristics**.
     The order of exploration is already known before the search phase.
   - **Dynamic heuristics**.
     The order of exploration is decided dynamically during the search phase.

How can we design search heuristics?
Assuming a feasible problem, it's very common to choose variables and values that are likely to yield a solution. However, first of all there is no guarantee of feasibility and secondly sometimes we get stuck in a kind of unfeasible sub-tree, making the CSP really difficult. Therefore, heuristics are designed in a such way that if we get stuck in a wrong solution they help us to get out from this unfeasible sub-tree as soon as possible.

Almost of the heuristics are designed on the **fail-first principle**: try first where you are most likely to fail so as to backtrack immediately. Of course, we never know if a particular sub-tree is part of the final solution at the beginning. Therefore, heuristics try to take some trade-off, defining a sort of strategy to find out the solution. We have two different approaches divided as follows:

- Choose next the **variable** that is most likely to cause failure.
- Choose next the **value** that is most likely to be a part of a solution.

Most of the heuristics rely on the variables choice, formally named **Variable Ordering Heuristics (VOHs)**: if we are in an unfeasible solution, the algorithm will try all the possible values to discover at the end its unfeasability.

4.6 Generic dynamic VOHs based on FF

We have several types of dynamic heuristics based on the fail-first principle:

- **Minimum domain**.
  We choose the decision variable X_i with the **minimum domain size**, $min(|D(X_i)|)$.
  This strategy is mainly used to minimize the search tree size.

Minimum domain

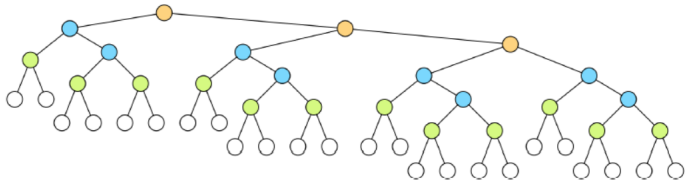
e.g. Consider the following decision variables order:

$$\langle X_1, X_2, X_3 \rangle$$

and their domains:

$$X_1 \in \{0, 1, 2, 3\}, X_2 \in \{0, 1, 2\}, X_3 \in \{0, 1\}$$

We will get a search tree of this size. Of course, changing the order of the decision variables the size would be the same.



Now we consider a total different order:

$$\langle X_3, X_2, X_1 \rangle$$

The size of the new search tree is the same as before, but we can observe that what really changes is the dimension of the sub-trees. Following a minimum domain choice, the propagation effect would be much stronger with the second ordering than the first one.

- **Most constrained**.
  We choose the variable involved in the **most number of constraints**. This strategy is mainly used to maximize the constraint propagation.
- **Combination**.
  We choose a variable that satisfies **both** previous strategies. Usually we take the variable that minimizes tha ratio:

$$\frac{domain\ size}{degree}$$

where:

- *domain size* → size of the decision variable domain
- *degree* → number of constraints in which the variable is involved

4.7 Weighted degree heuristic

In this type of heuristic we assign a **weight** to each constraint, initially set to 1. The total variable weight comes from the summation of the constraints in which it is involved.

How do we assign weights to constraints?
As we already said, initially we set each of them to 1; as the search proceeds and constraint propagation takes place for a whatever constraint c, the weight of that constraint c, formally $w(c)$, is going to be incremented by 1 if that constraint c fails.

The **weighted degree** $w(X_i)$ of a decision variable X_i is described as follows:

$$w(X_i) = \sum_{c \text{ s.t. } X_i \in X(x)} w(c)$$

This typology of constraint is very similar to **combination strategy** introduced before, since we choose the variable with the minimum ratio:

$$\frac{|D(X_i)|}{w(X_i)}$$

Note: in this case we talk about the weighted degree and not about the degree seen before in the **most constrained** dynamic strategy. By the weighted degree we are defining how much is crucial a whatever constraint c, while in the other hand by the most constrained strategy we are counting how many times a variable X_i is involved in the set of constraints.

4.8 Heavy tail behaviour

Another thing we can do to improve our search performance comes from the **heavy tail behaviour**. Assuming a Constraint Satisfaction Problem that requires some inputs (*e.g. in the n-queens problem the number of queens is an input parameter*), its complexity depends from the size of the input parameters, even though the model still always the same.

We often observe some exceptionally hard instances that take exceptionally longer time to solve. Of course, solving an instance of the n-queens problem within 1000 of queens takes much more time than solve it with 4 queens.

The thing is: **why this is happening**?
The main observation is that: this occurs not because any instance of the problem is itself difficult to solve by nature, but because we are playing with different parameters of the solver. For some combinations between the instances of the problem and the solver parameters, we get trapped into an exponential subtree, making the problem difficult to solve.

At the beginning of the search process all the heuristics used are unreliable: if we make a mistake early during search, we get stuck in a subtree. Therefore, the conclusion is that we don't really know how heuristics work, such mistakes or wrong decisions seem to be totally **random**.

In order to avoid these random mistakes we can do:

- **Randomization**.
  Since these mistakes are random, why don't we introduce **randomization** in our solvers also? By randomization we can kill this effect associated to randomness. For instance we could:
  1. Pick some variables and/or values randomly.
  2. Break ties randomly.
- **Restarting**.
  Restart the search process can be very useful when the algorithm keeps backtracking without returning us anything. For sure, to be effective we have to search in a different way anytime we restart the process. Searching anytime differently we also introduce:
  1. Randomization.
  2. Learning from the accumulated experiences of previous runs, or in other words previous runs can be effective for the following ones.
- **Randomization + Restarting**.
  We could also combine together the previous strategies, deleting the huge variance in solver performance.

4.9 Restart strategies

Assuming a variable L as the target value of resources we can use, we have different ways to restart the search process:

- **Constant restart**.
  Everytime L is totally consumed we restart the search process.
- **Geometric restart**.
  It introduces another parameter α, the resource limit now becomes $\alpha^n \times L$, where $n \in N$. Everytime the new limit $\alpha^n \times L$ is consumed, we restart the search process and we increment the exponential value of α (α^{n+1}).
- **Luby restart**.
  Luby restart is the more effective approach for any kind of Constraint Satisfacion Problem. We restart the search process after $s[i] \times L$ resources are consumed, where $s[i]$ is the ith number in the **Luby sequence**.

Note: the **domain weighted heuristic** (**domWdeg**) is quite effective within **restart strategy**.

4.10 Constraint Optimization Problem (COP)

So far, we focused on finding a solution that satisfies all the set of constraints, but also we have an objective function to discover the **best solution** possible. Formally, a Constraint Optimization Problem is defined by the following sequence:

$$\langle X, D, C, f \rangle$$

where:

- $X \rightarrow$ set of decision variables
- $D \rightarrow$ domains of the decision variables
- $C \rightarrow$ set of constraints between the decision variables
- $f \rightarrow$ formalization of the optimization criterion as an objective function

How does the solver behave to find the best solution given a specific objective function?
It uses a particular algorithm called **branch and bound algorithm**. This algorithm discovers the optimal solution (COP) solving a sequence of CSPs. Everytime the algorithm solves a Constraint Satisfaction Problem of the previous list, we incorporate a new constraint, named **bounding constraint**, before solving the following one. More formally, it follows three main steps:

1. Each time a feasible solution is found, **posts** a new bounding constraint which ensures that a future solution must be better than it.
2. **Backtracks** to the last decision and looks for a new solution with the additional bounding constraint, using the same search tree.
3. **Repeats** the same process until it becomes unfeasible, the last solution found would be the optimal one.

Optimal map colouring

e.g. What is the minimum number of colors to colour a map?

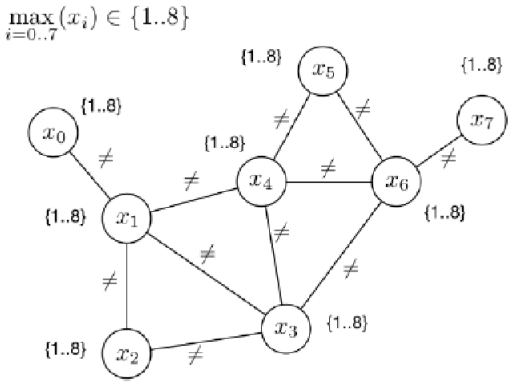
Decision variables and domains.
Every decision variable would be a region and any single domain value represents a different color. More formally, X_i with $i \in \{1, \ldots, n\}$ are the decision variables, while {1, . . . , n} it's the domain.

Constraints.
- $X_i \neq X_j$ for each neighboring region i and j.

Objective function.
- $f(X) = max(X_i)$ where:
  - $X_i \rightarrow$ number of colors used
  - $max(\ldots) \rightarrow$ maximize the number of colors used

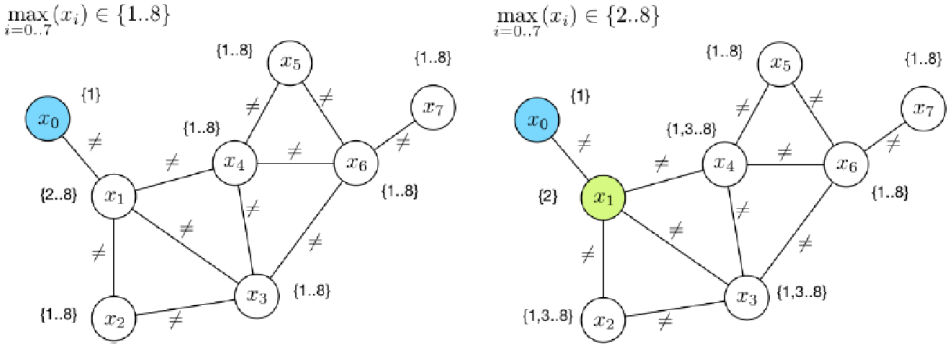
Objective.
- $min(f(X))$

In order to obtain the minimum number of colors we have to minimize the objective function f.

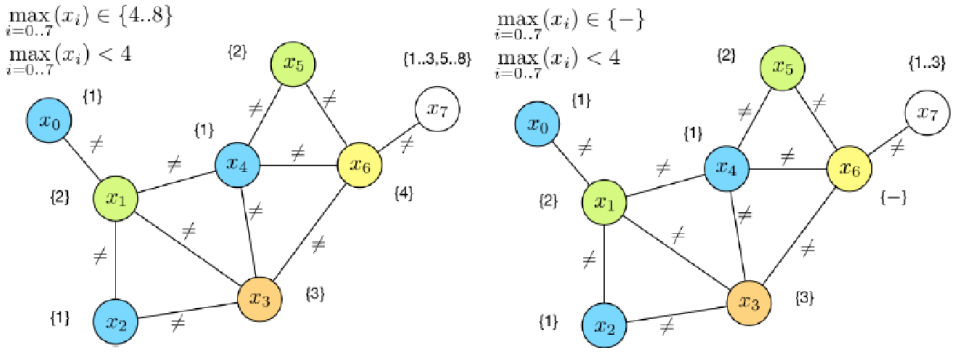


The same problem can be expressed as a graph, where links between variables represent neighboring regions. What makes this problem difficult is the optimality, namely find the minimum number of colors. The algorithm works as follows:

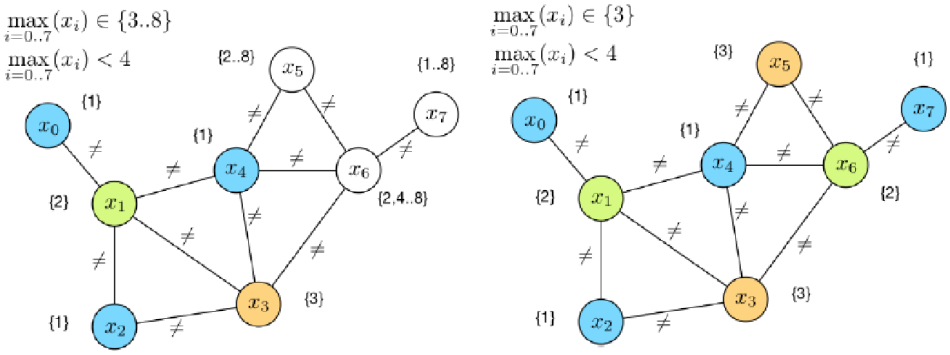
1. Pick a variable.
2. Assign a color to the variable.
3. Propagate the constraints involved.



We maximized the number of colors, finding a feasible solution for the first Constraint Satisfaction Problem. Now we backtrack to the last decision done and we add an additional bounding constraint, $max_{i=0...7}(X_i) < 4$. We have a new constraint to satisfy and, as we can observe from the image above, we fail: we cannot choose any value less than 4.



Therefore, we go one level higher, doing another backtrack step, until we reach a potential feasible resolution of the problem.



4.11 Complementary strengths

Let's take a look about the things seen so far.

CP.

+ A generic and complete approach with a focus on constraints and feasibility. We can apply it to model any optimization problem and giving to it enough time it will explore the whole search tree, finding a feasible solution.
+ Easy modelling and control of search.

- Cannot scale to large optimization problems. The search tree can be quite huge, making the search process very challenging.
- Poor in optimization with loose bounds on the objective function.

Heuristic search.

+ Scales to large optimization problems. For instance, local search or meta-heuristics are the kind of techniques that do no search over the whole search tree, but in portions of it.
+ Effective in finding good-quality solutions quickly.

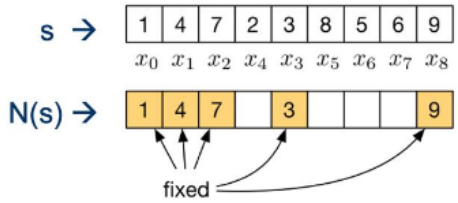
- Finding an initial solution and exploring a large neighborhood can be challenging.
- Neighborhoods are problem-specific.

Why don't we combine both of them, trying to explore exhaustively the search tree as CP and improving the solution in a more effective way like heuristics?

4.12 Large neighborhood search

The combination of CP and search heuristics gives us the **large neighborhood search**. Large neighborhood search is a hybrid method able to mix together the benefits coming from the previous strategies. The main idea behind the hybrid approach is described in the following steps:

1. Use CP to find an initial solution s, exploring a big neighborhood.
2. Use heuristics to explore different neighborhoods of the initial solution s. At the same time, this step is divided into:
   (a) Keep the assignments already done inside the solution s.
   (b) Look for the assignments of the remaining decision variables.



As shown in the image above, s is the initial solution, where every decision variable has its own value. Maybe, attempting to improve the initial solution s, some variables might not have a correspondence anymore. Therefore, we look for new assignments exploring neighborhoods of the initial solution.

We get several advantages coming from the hybrid approach, but the most valuable ones are:

- Efficient neighborhood exploration.
- Large neighborhood search is easier to develop than heuristic search.

Chapter 5

SAT

The acronym **SAT** comes from the word **satisfiability**. A SAT problem is a well-known decision problem: given a propositional formula is satisfiable or not?

Before moving on, we define formally what is a propositional formula and some properties about it.

Definition

A formula in propositional logic is composed by boolean variables and logical operators, which are divided into:

- $\neg \rightarrow$ negation
- $\vee \rightarrow$ disjuction
- $\wedge \rightarrow$ conjuction
- $\rightarrow\rightarrow$ implication
- $\leftrightarrow\rightarrow$ double implication

For example, the following formulas:

$$(p \vee q)$$

$$\neg(p \wedge q) \vee (r \vee t)$$

are propositional formulas.

Definition

A propositional formula is **satisfiable** if and only if it is possible to find a truth assignment to the variables that yields the formula true.

Example

e.g. Examples of satisfiability and unsatisfiability.

For $p = T$ and $q = F$, the following propositional formulas are:

$$(p \vee q) \wedge (\neg p \vee \neg q) \wedge (p \vee \neg q) \rightarrow \text{ SAT}$$

$$(p \vee q) \wedge (\neg p \vee \neg q) \wedge (p \vee \neg q) \wedge (\neg p \vee q) \rightarrow \text{ UNSAT}$$

We evaluate each time the whole formula. When we prove a formula we said that it is **SAT** (*satisfiable*) or whether we are not able to satisfy it we said that it is **UNSAT** (*unsatisfiable*).

5.1 Why SAT?

Why do we care about SAT? SAT is the first proven **NP-complete** decision problem (*NP is the complexity class between the tractable and intractable problems*).

All problems in the complexity class NP, including a wide range of decision and optimization problems, are reducible to SAT in polynomial time: if we can reduce those problems in polynomial time to a SAT problem, we can prove that those problems are at most difficult to solve as SAT. Solving a SAT problem we could determine that a problem is NP-complete.

5.2 Brute force approach

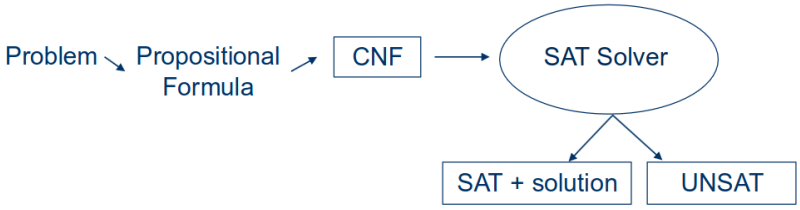
Given a boolean formula with n variables, how can we assign truth values to these variables?

We could try to use the **brute force approach**: build a truth table with 2^n different entries and search for the most feasible boolean assignment. This approach will always work but it's exponential in size according to the number of the variables, impractical for huge boolean formulas.

We need a dedicated SAT solver for checking the satisfiability of a formula and finding a satisfying assignment.

5.3 Problem solving with SAT

SAT solver is described by the following pipeline.



Before moving on, let's take a look at the image above:

1. Problem $\rightarrow$ Propositional formula: translation of the original problem to a boolean formula.
2. Propositional formula $\rightarrow$ CNF: conversion of the boolean formula to an intermediate form, called Conjuctive Normal Form (*CNF*), conjuction of disjuctions.
3. CNF $\rightarrow$ SAT solver: CNF boolean formula given to the SAT solver.
4. SAT solver $\rightarrow$ (SAT + Solution) $\vee$ (UNSAT): SAT solver gives us the final solution of the original problem.

The first step (*Problem* $\rightarrow$ *Propositional formula*) of the SAT solver pipeline is called the **encoding** phase, while the second part from *Propositional* $\rightarrow$ *CNF* to *SAT solver* $\rightarrow$ *Final answer* is the **solving** phase.

5.4 Formula satisfiability v.s. validation

Definition

A formula f is **satisfiable** if there is an assignment of values to the variables under which f evaluates to T.

Note: satisfiability is about finding a solution to a set of constraints, where each constraint is expressed as a propositional formula.

Definition

A formula f is **valid** if f always evaluates to T for any assignment of values to the variables.

A formula f is **valid** also if and only if $\neg f$ is UNSAT.

If we give to the SAT solver the last statement reported, telling us that it is **UNSAT**, it really means that originally that problem is **SAT**. Let's take a look at an example for a better understanding.

Example

e.g. Proving the de Morgan's law.

The de Morgan's law says:

$$\overline{(a \wedge b)} \equiv \neg a \vee \neg b$$

Proving this equivalance can be done exploiting the truth tables, searching for a feasible assignment of truth values. As we already know, this approach does not scale very well for more complex problems, since it grows exponentially in size.

Given $a \wedge b \equiv \overline{\overline{a \wedge b}}$, the same equivalance can be expressed in the following way:

$$a \wedge b \leftrightarrow \overline{(\neg a \vee \neg b)}$$

nothing changes, we just express the original equivalance in a different way.

This last step helps us to prove the de Morgan's law. Giving the negated formula

$$\neg(a \wedge b \leftrightarrow \overline{(\neg a \vee \neg b)})$$

to the SAT solver and proving that it is unsatisfiable by any possible assignment, we can derive that the original formula, $f_1 \leftrightarrow f_2$, is **valid**.

This is the chore mechanism of the SAT solver.

5.5 Encoding for validation

Example

e.g. Encoding a logic puzzle.

We consider the following clues:

1. Good-natured tenured professors are dynamic.
2. Grumpy student advisors play slot machines.
3. Smokers wearing a cap are phlegmatic.
4. Comical student advisors are professors.

5. Smoking untenured members are nervous.
6. Phlegmatic tenured members wearing caps are comical.
7. Student advisors who are not stock market players are scholars.
8. Relaxed student advisors are creative.
9. Creative scholars who do not play slot machines wear caps.
10. Nervous smokers play slot machines.
11. Student advisors who play slot machines do not smoke.
12. Creative good-natured stock market players wear caps.

Can we conclude that **no student advisor is smoking**?

In order to answer the previous question, we must give a name for every notion to be formalized.

Name	**Meaning**	**Opposite**
a	good-natured	grumpy
b	tenured	
c	professor	
d	dynamic	phlegmatic
e	wearing a cap	
f	smoke	
g	comical	
h	relaxed	nervous
i	play stock market	
j	scholar	
k	creative	
l	play slot machine	
m	student advisor	

Defined a name for every notion, now we are able to express natural language clues in propositional formulas. For instance, the first clue:

Good-natured tenured professors are dynamic

becomes:

$$(a \wedge b \wedge c) \rightarrow d$$

where:

- $a \rightarrow$ good-natured
- $b \rightarrow$ tenured
- $c \rightarrow$ professors
- $d \rightarrow$ dynamic

Therefore, the whole set of clues becomes as follows:

1. $(a \wedge b \wedge c) \rightarrow d$
2. $(\neg a \wedge m) \rightarrow l$
3. $(f \wedge e) \rightarrow \neg d$
4. $(g \wedge m) \rightarrow c$
5. $(f \wedge \neg b) \rightarrow \neg h$
6. $(\neg d \wedge b \wedge e) \rightarrow g$
7. $(\neg i \wedge m) \rightarrow j$
8. $(h \wedge m) \rightarrow k$
9. $(k \wedge j \wedge \neg l) \rightarrow e$
10. $(\neg h \wedge f) \rightarrow l$
11. $(l \wedge m) \rightarrow \neg f$
12. $(k \wedge a \wedge i) \rightarrow e$

After this process of encoding, the original question now is:

$$\neg(m \wedge f)?$$

Giving to the SAT solver the previous question and the set of propositional formulas listed before, it proves that the formula $\neg(m \wedge f)$ is UNSAT, so we conclude saying that there is no student advisor that is smoking given this set of clues.

5.6 Encoding for satisfaction

We can use SAT for many purposes, like validation, satisfiability of properties but also for Constraint Optimization Problems.

Example

e.g. Optimal map colouring.

Can we assign a color to the regions such that adjacent regions have different colors?

Boolean decision variables.
A boolean decision variable x_{ic} describes each possible color $c \in \{1, \ldots, k\}$ and region $v_i \in V$.

- $x_{ic} = T$ means that a region v_i is assigned to the color c.

Constraints.

- Each region gets at least one color.

$$\forall v_i \in V, x_{i1} \vee \cdots \vee x_{ik}$$

  More synthetically:

$$\forall v_i \in V, \bigvee_{1 \leq v \leq k} x_{ic}$$

  This new expression tells us the same thing as the original one, we can assume that between all the boolean decision variables x_{ic} we have a logical *OR*.

- Neighboring regions takes different colors.

$$\forall (v_i, v_j) \in E, \forall c \in \{1, \ldots, k\}, \neg(x_{ic} \wedge x_{jc})$$

  Or more synthetically:

$$\forall (v_i, v_j) \in E, \bigwedge_{1 \leq c \leq k} \neg(x_{ic} \wedge x_{jc})$$

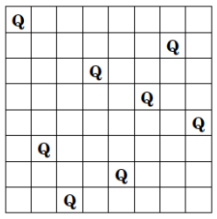
  For each pair of neighboring regions, they cannot share the same color.

5.7 N-Queens in SAT

Example

e.g. N-Queens in SAT.

Can we place in a 8×8 board 8 queens without they attack each other?



Boolean decision variables.
A boolean decision variable p_{ij} describes each possible position on the board.

- $p_{ij} = T$ means there is a queen on row i and column j.

Constraints.

- Exactly one queen on each row.

  The same condition can be expressed as the combination of two different constraints, which are:

  – **At least one** queen on a row i.

  $$\forall i \in N, p_{i1} \vee \cdots \vee p_{in}$$

  More synthetically:

  $$\forall i \in N, \bigvee_{1 \leq j \leq n} p_{ij}$$

  – **At most one** queen on a row i.

  $$\forall i \in N, \forall 0 < j < k \leq n, \neg(p_{ij} \wedge p_{ik})$$

  Or more synthetically:

  $$\forall i \in N, \bigwedge_{0 < j < k \leq n} \neg(p_{ij} \wedge p_{ik})$$

  By **at least one** constraint we avoid that no queen is placed in a row, while by **at most one** constraint we force the solver to put only one queen for each row.

- Exactly one queen on each column.

  Column constraints are the same as the row constraints with i and j swapped.

Therefore, we can combine together row and column constraints satisfying the generic form of the problem.

- **At least one** queen on each row and column.

$$\bigwedge_{1 \leq i \leq n} \bigvee_{1 \leq j \leq n} p_{ij}$$

$$\bigwedge_{1 \leq j \leq n} \bigvee_{1 \leq i \leq n} p_{ij}$$

- **At most one** queen on each row and column.

$$\bigwedge_{1 \leq i \leq n} \bigwedge_{0 < j < k \leq n} \neg(p_{ij} \wedge p_{ik})$$

$$\bigwedge_{1 \leq j \leq n} \bigwedge_{0 < i < k \leq n} \neg(p_{ij} \wedge p_{ik})$$

The reasoning about the diagonal constraints is a little bit different than before. **At most one** constraint is necessary, since two queens cannot stay in the same diagonal. However, can we say the same thing for **at least one** constraint? We do not need at least one reasoning, since given a diagonal there might not be a queen.

Additionally, we can detect if two or more queens share the same diagonal when the sum or the difference between their row and column indeces are the same.

- p_{ij} and $p_{i'j'}$ are on the same diagonal if and only if:

$$i + j = i' + j' \vee i - j = i' - j'$$

- **At most one** queen on each diagonal.

$$\bigwedge_{1 \leq i < i' \leq n} \bigwedge_{j,j' : i+j = i'+j' \vee i-j = i'-j'} \neg(p_{ij} \vee p_{i'j'})$$

  This last constraint will take in account only those indeces such that the sums or the differences are the same, meaning the same diagonal.

Generally, the SAT solver stops after the first solution given, so we lose the possibility to look at all the possible solutions.

How to find all solutions?
After finding a solution, add the negation of it to the formula, and repeat this until the formula is UNSAT.

5.8 Arithmetics in SAT

The following example shows us that encoding can be quite complex in SAT, since we have to convert everything in a propositional formula.

Example

e.g. Arithmetics in SAT.

$$7 + 21$$

How do we encode this simple operation in SAT solving?

We have to design a way such that we are able to encode three main things:
1. Number 7.
2. Number 21.
3. The meaning of sum two numbers together.

We could represent numbers in base 2 via n binary variables, taking values $T = 1$ and $F = 0$. Given a whatever number a in base 10, composed by the following digits $a_{n-1} \dots a_0$, in base 2 becomes:

$$a = \sum_{i=0}^{n-1} a_i * 2^i \;\; a_i \in \{0, 1\}$$

Decision problem.
Given a and b, find d satisfying $a + b = d$.

Boolean decision variables.
- $a_{n-1} \dots a_0 \rightarrow$ digits of the first number
- $b_{n-1} \dots b_0 \rightarrow$ digits of the second number
- $d_{n-1} \dots d_0 \rightarrow$ digits of the final result
- $c_{n-1} \dots c_0 \rightarrow$ carry digits. When we add two binary numbers equal to 1, we carry on the additional digit to the next column

For instance, the sum of $7 + 21$ is described by the image above:

$c \rightarrow$	0	0	1	1	1	0
$a = 7 \rightarrow$		0	0	1	1	1
$b = 21 \rightarrow$		1	0	1	0	1
$d = 28 \rightarrow$		1	1	1	0	0

The third digits of the numbers a and b are both equal to 1, therefore, after the sum, we carry the additional digit to the fourth column.

Through these decision variables, we get four different relationships. Assuming the following equivalance:

$$a + b + c = d$$

the relationships between these boolean variables are divided into:
1. $0 + 0 + 0 = 0$, we carry on $c = 0$
2. $0 + 0 + 1 = 1$, we carry on $c = 0$

3. $0 + 1 + 1 = 1$, we carry on $c = 1$
4. $1 + 1 + 1 = 1$, we carry on $c = 1$

The entire system of equivalances described previously must be taken in account and translated in a specific constraint.

Constraints.

- Compute d_i from the right to left starting from $c_0 = 0$

  The constraint itself is defined by other three subprocesses, like:

  1. **Compute the result digit by digit**.

  $$d_i = a_i + b_i + c_i \bmod 2, \text{ for } i = 0, \ldots, n-1$$

  2. **Compute the digit of the carry boolean decision variable**.

  $$c_{i+1} = 1 \leftrightarrow a_i + b_i + c_i > 1, \text{ for } i = 0, \ldots, n-1$$

  3. **Set the final and the initial digits of the carry boolean decision variable to** 0.

  $$c_n = 0 \text{ and } c_0 = 0$$

  As we already said, the SAT solver works only with propositional formulas. Therefore, we need to convert these subprocesses into their propositional counterparts. They become:

  1. $$a_i \leftrightarrow b_i \leftrightarrow c_i \leftrightarrow d_i$$
  2. $$c_{i+1} \leftrightarrow (a_i \wedge b_i) \vee (a_i \wedge c_i) \vee (b_i \wedge c_i)$$
  3. $$\neg c_n \wedge \neg c_0$$

Let f be the conjuction of all these formulas. To compute the sum $a + b = d$, the final propositional formula is:

$$f \wedge \bigwedge_{i=0}^{n-1} [\neg] a_i \wedge \bigwedge_{i=0}^{n-1} b_i$$

where:

- $f \rightarrow$ conjuction of all the previous propositional formula
- $\bigwedge_{i=0}^{n-1} [\neg] a_i \rightarrow$ expression of the first number in base 2

- $\bigwedge_{i=0}^{n-1}[\neg]b_i \rightarrow$ expression of the second number in base 2

For instance, when $a = 13$ and $b = 7$, we have:

$$f \wedge \underbrace{\neg a_1 \wedge a_2 \wedge a_3 \wedge \neg a_4 \wedge a_5}_{a=13=01101} \wedge \underbrace{\neg b_1 \wedge \neg b_2 \wedge b_3 \wedge b_4 \wedge b_5}_{b=7=00111}$$

for which a SAT solver would return us:

$$\underbrace{d_1 \wedge \neg d_2 \wedge d_3 \wedge \neg d_4 \wedge \neg d_5}_{d=20=10100}$$

If d does not fit in n digits, c_n will be forced to be 1, and the result will be UNSAT. A quick solution would be to expand the formula with leading bits in a and b set to 0.

5.9 Problem solving in SAT

So far, we focused only on the **encoding phase**: the way we translate the problem from the natural language expression to a propositional formula. Following the SAT solver pipeline, we need to understand also the **solving phase**, starting from the CNF propositional formula.

In order to understand the CNF formula we introduce some terminology.

Literal

Refers either to a boolean variable p or to its negation $\neg p$.

Clause

A clause is a disjunction of literals. It can be falsified with only one assignment to its literals, where all literals are assigned to F.

CNF propositional formula

A propositional formula f is in CNF if and only if it is a conjuction of clauses, or a conjuction of disjunction of literals.

For instance:

$$f = C_1 \wedge C_2 \wedge C_3$$

Formally, the same thing can be expressed as:

$$\bigwedge_i \bigvee_j l_{ij}$$

A CNF propositional formula is satisfiable if there exists an assignment satisfying all clauses, otherwise it will be unsatisfiable (*each clause of the more generic formula can be satisfied putting one literal to true*).

5.10 Resolution

Given a formula represented in CNF, the SAT solver uses a basic method to solve it. This method is called **resolution**, and it works as follows: from the given clauses, it derives new clauses, with the aim of deriving the empty clause $\perp$, also named **contradiction**, proving the unsatisfiability of the propositional formula.

Resolution rule

Given the clauses of the following shape:

$$p \vee W \text{ and } \neg p \vee W$$

we can derive $V \vee W$.

We are able to do this *semplification* according to two different observations:

1. The original formulas share a propositional variable, but in one clause is positive while in the other one we get it in a negative form.
2. We don't care about the truth value given to p. For any assignment we give to p, V or W must be true.

Sometimes a clause consists of a single literal l, then the **resolution rule** allows to remove the literal $\neg l$ from a clause containing exactly $\neg l$.

Unit resolution

When V or W in

$$\frac{p \vee V, \neg p \vee W}{V \vee W}$$

are empty, we have:

$$\frac{p, \neg p \vee W}{W} \text{ or } \frac{p \vee V, \neg p}{V}$$

Example

e.g. Resolution mechanism.

Prove that the CNF consisting of the following five clauses is UNSAT, so it reaches contradiction.

1. $p \vee q$
2. $\neg r \vee s$
3. $\neg q \vee r$
4. $\neg r \vee \neg s$
5. $\neg p \vee r$

Solution.
Therefore, we have to match these different clauses until we reach a contradiction, defined in almost of the cases by the bottom symbol $\perp$. For instance, we can start combining together the first and the third clauses as follows:

$$\frac{p \vee q, \neg q \vee r}{p \vee r}$$

So, now we have derived a new clause and we add it to our previous list:

1. $p \vee q$
2. $\neg r \vee s$
3. $\neg q \vee r$
4. $\neg r \vee \neg s$
5. $\neg p \vee r$
6. $p \vee r$

We continue to match the clauses until we reach a contradiction. The final result will be as below:

1. $p \vee q$
2. $\neg r \vee s$
3. $\neg q \vee r$
4. $\neg r \vee \neg s$
5. $\neg p \vee r$
6. $p \vee r$
7. r
8. s
9. $\neg r$
10. $\perp$

The resolution process is perfect when we have to prove the unsatisfiability of a formula. However, almost of the times, this is not enough, since we could need a

satisfying solution. Therefore, we need to combine this mechanism with something else.

5.11 DPLL

DPLL is an algorithm to establish the satisfiability or unsatisfiability of a CNF propositional formula.

The basic idea of this algorithm is divided into three main steps:

1. Apply **unit resolution** as long as possible. At some point we will not able to derive any other clause, so we integrate search processes, something similar we did with Constraint Programming.
2. **Choose** a boolean decision variable p, introducing inside the search tree the cases p and $\neg p$.
3. After we introduced the positive and negative cases, we **go recursively** until we reach the satisfiability or unsatisfiability condition.

DPLL pseudocode

```
procedure DPLL(X)
    X = unit-resol(X)
    if X = ∅ then
        return(sat)
    end if
    if ⊥ ∉ X then
        choose variable p ∈ X
        DPLL(X ∪ {p})
        DPLL(X ∪ {¬p})
    end if
end procedure
```

Before moving on, let's take a look at the previous snippet of code. The main steps of the DPLL algorithm can be described as follows:

1. First of all, we apply **unit resolution** as long as possible.
2. We check if we have reached the **contradiction**, if so we return the satisfiability condition.
3. If we didn't come up with the bottom or the empty set, we add a new boolean variable inside the description of the problem.
4. Finally, we augment the original set of propositional formulas, adding the positive and negative cases of boolean variable chosen.

DPLL and Constraint Programming are very similar paradigms:

1. Like Constraint Programming, we check during the resolution if we could reach a contradiction; if so, we backtrack choosing another variable or assigning another boolean value.
2. Both of them are complete methods. Giving to them enough time, they always return a final answer that might be either positive or negative.
3. Efficiency strongly depends on the choice of the variable.

Example

e.g. DPLL resolution process.

1. $\neg p \vee \neg s$
2. $\neg p \vee \neg r$
3. $\neg q \vee \neg t$
4. $p \vee r$
5. $p \vee s$
6. $r \vee t$
7. $\neg s \vee t$
8. $q \vee s$
9. $q \vee \neg r$

This set of propositional formulas doesn't allow any unit resolution, since there is no boolean variable alone. Therefore, we need to choose a variable and add it inside this set, defining its positive or negative value.

Assuming we choose the boolean variable p and we add it inside the resolution in its positive form $p = T$, the mechanism goes as follows:

1. $\neg s$ (1)
2. $\neg r$ (2)
3. q $(\neg s, 8)$
4. t $(\neg r, 6)$
5. $\neg t$ $(q, 3)$
6. $\perp$ $(t, \neg t)$

Note: after we add in the set $p = T$ then (1) and (2) propositional formulas of the original list must be exactly true, therefore both $\neg s$ and $\neg r$ must be equal to false.

As we can observe, we have reached a contradiction, so the problem is unsatisfiable *(UNSAT)*. Now we try with its counterpart, adding to the original list of propositional formulas $\neg p$.

1. $r(4)$
2. $s(5)$
3. $q(r, 9)$
4. $t(s, 7)$
5. $\neg t(q, 3)$
6. $\perp(t, \neg t)$

Again, we have reached a contradiction, therefore the whole problem is UNSAT.

5.12 Implementation of DPLL

We know how the DPLL algorithm works in general terms, a mechanism where **unit resolution** and **search** are combined together.

Now we move to the implementation of the algorithm. A direct implementation of the pseudocode seen so far would make a copy of the *CNF* X at every recursive call (X *is the propositional formula*), potentially going out of memory very quickly.

What really happens is that somehow the implementation of the algorithm mimics the DPLL approach described previously, using a main data structure are performed the following operations:

1. Unit resolution.
2. Case Analysis.
3. Backtrack.
4. Fail.

So, basically we mimic the four main operations listed before inside a particular data structure, usually called M. The basic idea is: keep track of a list M of literals (*boolean varibles or their negations*) that have been decided and derived during the execution of DPLL.

More formally, the algorithm behaves following some **rules**:

1. M starts empty.
2. M is extended when:
   (a) **UnitPropagate**, a literal is derived by unit resolution;
   (b) **Decide**, a new case analysis starts.
3. M is repaired when a contradiction is found:
   (a) **Backtrack**, go back to the last decision, remove everything behind the last decision, negate the decision, and continue with a new decision;

(b) **Fail**, otherwise, when it is not possible to backtrack, fail.

Additionally, it uses a little bit of formal definitions about propositional logic, which are:

- A literal l holds in M ($M \models l$) if and only if l occurs in M.
- A clause C yields contradiction ($M \models \neg C$) if and only if for every literal l in C, we have, $M \models \neg l$. It simply says: if all the literals of C are present in M in their negated form, we cannot satisfy anymore the clause.
- l is *undefined* in M if and only if neither $M \models l$ nor $M \models \neg l$, so l is not in the list M.
- A decision of a literal, described as l^d (*there isn't anything about mathematics, it expresses when a decision about a boolean variable was taken*), originates from a decision in the DPLL algorithm.

5.13 Four rules

Let's take a look at the four rules more precisely. As we already know, they are used in order to mimic the real behavior of the DPLL algorithm, since its direct implementation it's not so efficient.

1. **UnitPropagate**.
   Mimics the generation of a new unit clause in DPLL. In practice, what the process really does is to extend the list M with some new literals, already derived, incorporating them inside the entire formula. Formally, this rule can be described as:
   $$M \to Ml$$
   This occurs if l is **undefined** in M, not in the list, and the CNF formula contains a clause $C \vee l$ satisfying $M \models \neg C$.
2. **Decide**.
   Mimics the choice p in DPLL, when no UnitPropagate is possible. **Decide** rule does not need any propagation, it just selects which boolean variable to introduce and its truth value.
   $$M \to Ml^d$$
   This is possible if literal l is **undefined** in M.

   Note: l^d symbolizes the choice done, it does not rely on any other mathematical expression.
3. **Backtrack**.
   Mimics backtracking to the negation of the last decision in case a branch is

unsatisfiable. Formally, it's defined as follows:

$$Ml^dN \rightarrow M\neg l$$

where:
- $l^d \rightarrow$ decision done
- $N \rightarrow$ set of derivations obtained

This rule requires that $Ml^dN \models \neg C$ for a clause C in the CNF formula and N does not contain any literal coming from a decision.

If we reach a contradiction, we have to undo everything that came up from the last decision, deleting N and negating l (*the truth value of the literal l does not matter, it can be either true or false*).

Note: **Backtrack** and **UnitPropogate** are little bit different. By UnitPropagate we can do something: incorporating literals l to the formula we can still satisfy the original clause; while in Backtrack there is no way to satisfy the clause C.

4. **Fail**.
   Mimics the end of DPLL algorithm when every branch, and so the CNF formula, is unsatisfiable.

   $$M \rightarrow fail$$

   This happens if $M \models \neg C$ for a clause C in the CNF formula and M does not contain decision literals.

   In practice, we have a clause that cannot be falsified and M does not contain any decision d left to backtrack.

5.14 Example

Example

e.g. Mimicking DPLL algorithm.

Starting from the following list of clauses:
1. $\neg p \vee \neg s$
2. $p \vee r$
3. $\neg s \vee t$
4. $\neg p \vee \neg r$
5. $p \vee s$
6. $q \vee s$
7. $\neg q \vee \neg t$

8. $r \vee t$
9. $q \vee \neg r$

Define if the whole CNF propositional formula is satisfiable or unsatisfiable.

Resolution.

Rule	M

At the beginning of any possible resolution, the only rule that can be applied is the **Decide** one: pick one variable from the list and give to it a truth value.

Rule	M
Decide	p^d

After we decided the boolean variable and gave to it a truth value ($p = T$), we can try to set up the unit propagation, extending p incorporating new literals. We already know that p is set to true, so we have to check the other clauses attempting to give to each of them the correct boolean value.

In this case, we can observe:
1. $\neg(p = T) \vee \neg s \rightarrow \neg s$
4. $\neg(p = T) \vee \neg r \rightarrow \neg r$
8. $(r = F) \vee t \rightarrow t$
6. $q \vee (s = F) \rightarrow q$

The process goes as described in the list before, propagating unit until we reach the resolution of the problem or a contradiction. Additionally, it's important to state that the unit propagation does not involve only the boolean variable chosen, but also the ones linked to it.

Rule	M
Decide	p^d
UnitPropagate	$p^d\ \neg s\ \neg r\ t\ q$

Observing what we did so far, we have reached a contradiction, in particular the literals t and q are set to true but, however, the seventh clause $(\neg q \vee \neg t)$ requires at least one of them assigned to false. Therefore, we backtrack deleting everything we did from the last decision taken and negating the boolean variable choosen.

Rule	M
Decide	p^d
UnitPropagate	$p^d\ \neg s\ \neg r\ t\ q$
Backtrack	$\neg p$

After the Backtracking rule, we can set up a new unit propagation process, trying to reach the satisfiability of the problem.

In the negative case, we can state:

2. $(p = F) \vee r \rightarrow$ r
5. $(p = F) \vee s \rightarrow$ s
9. $q \vee \neg(r = T) \rightarrow$ q
3. $\neg(s = T) \vee t \rightarrow$ t

Rule	M
Decide	p^d
UnitPropagate	$p^d\ \neg s\ \neg r\ t\ q$
Backtrack	$\neg p$
UnitPropagate	$\neg p\ r\ s\ q\ t$

However, we have reached again the same contradiction as before. Therefore, the final conclusion is the unsatisfiability of the problem.

Rule	M
Decide	p^d
UnitPropagate	$p^d\ \neg s\ \neg r\ t\ q$
Backtrack	$\neg p$
UnitPropagate	$\neg p\ r\ s\ q\ t$
Fail	

Chapter 6

SAT II

DPLL algorithm is not always implemented, in real terms the mechanism is set up for mimicking the real behavior of the process, using four main rules:

1. Unitpropagation.
2. Decide.
3. Backtrack.
4. Fail.

Anyway, also the approximated algorithm suffers from two main inefficiencies:

1. **No learning**.
   - If we fail, the algorithm throws away all the work performed to conclude that the current **partial assignment** is bad when we reach a contradiction.
   - It revisits bad partial assignments that lead us to a contradiction due to the same root cause.
2. **Chronological backtracking**.
   - Backtracks at one previous level, even if it can be deduced that the current bad partial assignment is caused by another level.

6.1 Conflict-Driven Clause Learning (CDCL)

Therefore, from the previous observations, it has been decided to design another type of SAT solver, named **Conflict-Driven Clause Learning**. Whenever a contradiction happens, it is set a process composed by the following operations:

1. **Clause learning**.

- Augments the original CNF X propositional formula with an additional **conflict clause** that summarizes the root cause of the contradiction.
- From the point above, we will learn from the mistakes done and ignore huge sections of the search space that would be useless to explore.

2. **Backjumping**.
   - Based on the cause of the contradiction, skips the decisions and derivations that did not lead to conflict, going higher in the search tree.

Example

e.g. Dumb example.

Starting from the following list of clauses:

1. $\neg p \vee q$
2. $\neg r \vee s$
3. $\neg t \vee \neg u$
4. $\neg q \vee \neg t \vee u$

Define if the whole CNF propositional formula is satisfiable or unsatisfiable.

Resolution.

Rule	M
Decide	p^d
UnitPropagate	$p^d\ q$
Decide	$p^d\ q\ r^d$
UnitPropagate	$p^d\ q\ r^d\ s$
Decide	$p^d\ q\ r^d\ s\ t^d$
UnitPropagate	$p^d\ q\ r^d\ s\ t^d\ u$

Observing what we did so far, we have reached a contradiction. In particular, the literals t and u are set to true but, however, the third clause ($\neg t \vee \neg u$) requires at least one of them assigned to false. We backtrack to the last decision done t^d and setting the literal t to false ($\neg t$) we state the satisfiability of the problem.

Anyway, supposing we have a bigger CNF formula, composed by more than four clauses. Even though we define the literal t equal to false ($\neg t$), we cannot achieve the satisfiability of the problem: we backtrack to the last decision before t^d, namely r^d, setting its value to another truth value. Later, we will try again the decision t^d failing in the same conflicts as before.

Briefly, even if we backtrack changing truth values, we end always in the same

contradictions, failing exactly in the same way.

It would be nice to understand something more by a contradiction analysis.

For the contradiction between the literals t, u and the third clause $(\neg t \vee \neg u)$, the decision r^d has nothing to do with them. What we really want to understand are the clauses that contribute to the contradiction described in the example: literal u was derived using the fourth clause $(\neg q \vee \neg t \vee u)$ and the contradiction was found using the third clause.

For sure we fail for the third clause $(\neg t \vee \neg u)$, but also we reach a contradiction combining it with the fourth one (*it sets the boolean variable u to true. Therefore, there is a kind of relationship between the third and the fourth clauses*). The main idea is to apply directly the resolution process to these clauses getting a new **conflict clause**, obtained as follows:

$$5.\ \neg q \vee \neg t$$

The clause 5. is something we learnt after the failure, taking this new information we can do unit propagation in a different way. This newly conflict clause gives us a clue on where to **backjump**.

Example

e.g. Extending the same previous example.

Resolution.

Rule	M
Decide	p^d
UnitPropagate	$p^d\ q$
Decide	$p^d\ q\ r^d$
UnitPropagate	$p^d\ q\ r^d\ s$
Decide	$p^d\ q\ r^d\ s\ t^d$
UnitPropagate	$p^d\ q\ r^d\ s\ t^d\ u$
Backjumping	$p^d\ q\ \neg t$

6.2 Backjumping

Mimics backtracking to an earlier decision literal than the last one. Formally, this process is described as follows:

$$Ml^dN \rightarrow Ml'$$

This occurs if $Ml^dN \models \neg C$ for a clause C in the CNF and there is a clause $C' \vee l'$ derivable from the CNF such that $M \models \neg C'$ and l' is not defined in M.

It may seem quite difficult to understand, so let's take in account our previous example. The **derivable clause** $C' \vee l'$ allows us to derive the literal l' such that the entire propositional formula remains satisfiable. Roughly speaking, in our example would be:

$$C' \vee l' = \neg q \vee \neg t$$

where:

$$\underbrace{p^d\ q}_{M}\ \underbrace{r^d}_{l^d}\ \underbrace{s\ t^d\ u}_{N} \rightarrow \underbrace{p^d\ q}_{M}\ \underbrace{\neg t}_{l'}$$

We already know that C' is not satisfiable, so we need something like l' in order to make the entire formula set to true.

6.3 Backjumping clause

How does the SAT solver find out all these informations? First of all backjump is an unit propagation process on a derivable conflict clause, which is added to the original CNF. In order to be able to backjump the solver stores:

- Each decision.
- At every UnitPropagate step it stores the corresponding clause and the derived literal.

The whole thing is saved inside a particular data structure, named **implication graph**. At each contradiction reached, we investigate over this data structure to find out which are the clauses and the literals that play a role in the conflict.

6.4 Implication graph

The CDCL solver creates and handles a particular data structure, called **implication graph**, mainly used to save several information. They will be used to perform **Backjumping** activities when necessary.

An implication graph ($G = (V, E)$) is a directed acyclic graph that records the history of decisions and the resulting deductions derived during the unit propagation processes:

- Each node v, with $v \in V$, represents a decision, a derived literal or a conflict. We label any node by a certain decision level d, so for instance the node $k@d$ describes a contradiction reached at a specific level d of the graph.
- Every edge $v \xrightarrow{c_i} w \in E$ indicates that w is derived with unit propagation from the clause c_i with one of its literals being v (*therefore, v and w share at least one clause*).

Example

e.g. Implication graph.

Starting from the following list of clauses:

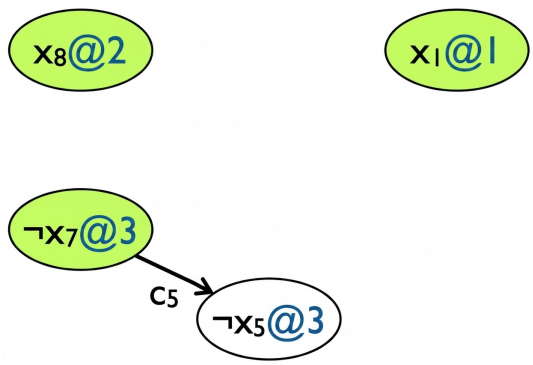
1. $\neg x_1 \vee x_2 \vee \neg x_4$
2. $\neg x_1 \vee \neg x_2 \vee x_3$
3. $\neg x_3 \vee \neg x_4$
4. $x_4 \vee x_5 \vee x_6$
5. $\neg x_5 \vee x_7$
6. $\neg x_6 \vee x_7 \vee \neg x_8$

Define if the whole CNF propositional formula is satisfiable or unsatisfiable.

Resolution.
We start randomly taking in account the boolean variable x_1, assigning a true value to it. Then, we create a node in our implication graph with a decision level d equal to 1, since it's the first level of the data structure.

We move to the second level, choosing the variable x_8, taking also for it a true value.

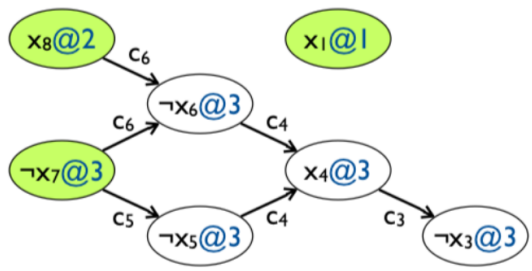
Now, we take in consideration at the third level the variable x_7, putting it to false. However, we can derive by unit propagation the term $\neg x_5$ from the fifth clause. This will be out first edge ever.



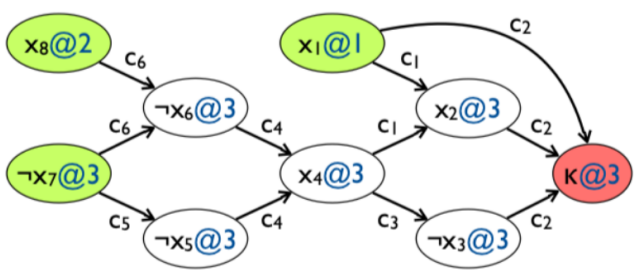
As we can observe from the image above, we have built a new node for x_5 boolean variable, maintaining the same decision level. $\neg x_5$ and x_7 are linked by an edge labelled by the clause that combines them together.

After we introduced x_7 node, we can also add $\neg x_6$ in the graph, derived from the sixth clause. The node $\neg x_6$ has the same decision level of x_7 and x_8, and it takes false as value.

Additionally, from the fourth clause we could derive x_4: it must be true in order to satisfy the propositional formula.



We keep going until we reach a contradiction or the satisfiability of the problem. However, taking these truth values we reach a contradiction: in the second clause $(\neg x_1 \vee \neg x_2 \vee x_3)$ both x_1 and x_2 are set to true and x_3 is set to false, reaching a conflict between them. Therefore, in the implication graph, we would create a node also for the conflict.



How can we derive new truth values from this graph? First of all, we apply the unit resolution between the first and second clauses. But, also the third clause is involved in the contradiction, so it's another entity to take in account in the resolution process. Anyway, at the same time we can state that $\neg x_4$ comes from the fourth clause and it has a direct influence on x_2 and x_3; so, maybe, also the fourth clause might be considered in the unit resolution mechanism. This type of reasoning keeps going until we cover the whole list of clauses, formally it can be expressed as follows:

- $\neg x_1 \vee x_3 \vee \neg x_4$ (c_1 and c_2)
- $\neg x_1 \vee \neg x_4$ (c_* and c_3)
- $\neg x_1 \vee x_5 \vee x_6$ (c_* and c_4)
- $\neg x_1 \vee x_6 \vee x_7$ (c_* and c_5)
- $\neg x_1 \vee x_7 \vee \neg x_8$ (c_* and c_6)

Note: with c_k with $k \in \{1, 2, 3, 4, 5, 6\}$ we mean the clauses of the original list. With c_* we mean the clause derived during the unit propagation.

Right now, from the unit propagation process we know that only x_1 and x_8 can be false and x_7 must be set to true. However, instead of going back so far until the beginnig, we could stop somewhere in the middle, since we have already found something new.

We got several back-steps from the previous list, but how can we choose the **more valid backjump**? In almost of the cases, we consider the definition of **unit implication point**.

Definition

An unit implication point is any node in the implication graph that is part of all the paths from the current decision literal ($l@d$) to the conflict ($k@d$).

In particular, the **first unit implication point** (*1UIP*) is the implication point which is closest to the conflict.

Normally, we will backjump at the decision x_7, but now we are gonna to come back on x_1 and x_4, since we learnt that both variables cannot be assigned to the same value.

Therefore, the first step is to add the new clause ($\neg x_1 \vee \neg x_4$) inside the list of clauses.

- $\neg x_1 \vee x_3 \vee \neg x_4$ (c_1 and c_2)

- $\neg x_1 \vee \neg x_4$ (c_* and c_3)
- $\neg x_1 \vee x_5 \vee x_6$ (c_* and c_4)
- $\neg x_1 \vee x_6 \vee x_7$ (c_* and c_5)
- $\neg x_1 \vee x_7 \vee \neg x_8$ (c_* and c_6)
- $\neg x_1 \vee \neg x_4$

Previously, we reached a contradiction by the following unit propagation:

$$x_1^d \; x_8^d \; x_7^d \; \neg x_6 \; \neg x_5 \; x_4 \; x_2 \; \neg x_3$$

however, as we discussed before, thanks to $\neg x_1 \vee \neg x_4$ we learnt that there was an unit propagation setting x_4 to the wrong truth value. Therefore, we can start again from the decisional node x_1:

$$x_1^d \; \neg x_4$$

6.5 Shortcomings of DPLL

One thing is about learning and an other thing is about the variable choice. So far, we chose variables randomly, but also in the SAT problems the variable choice is very important.

The choice of the variable is already integrated by the solvers, based on the concept of **activity-based search**. They concentrate on the variables causing conflicts, or, in other words, they focus on those variables that appear different times in conflict clauses and in new derived clauses. Each time a variable is involved in these kind of clauses solvers increment its weight associated to it. Therefore, we should select the boolean variable with **highest weight**.

6.6 Search restarts

SAT and CP problems have different similarities, like **restart**. Restart is a very effective activity for SAT solving, and typically it's done when the search for a solution does not progress while many clauses have been learnt.

Each time we restart the SAT solving the following benefits are achieved:

1. We get a different new CNF propositional formula.
2. We are able to randomize the variable selection heuristic, always based on the weights of the boolean variables.

3. Previously derived clauses and the activities influence the variable ordering heuristic.

6.7 Optimizing unit propagation

There are two basic rules to optimize unit propagation:

1. If all but one of a clause's literals are assigned to false and the remaining literal is unassigned, assign it to true.
2. If all clause's literals are assigned to false, return the **unsatisfiability** of the problem.

The main idea about the naive algorithm is about to inspect each clause and apply the two previous rules; it repeats this process until no new assignment are made. However, for more complex propositional formulas inspect each clause one by one cannot be feasibile, we need an efficient way to do so.

6.8 Watched literals

Any unit propagation process is performed following the **watched literals** behavior. Watched literals is the best known way to approximate the naive algorithm. The approximated algorithm is defined by the following steps:

- Watch two unassigned literals in every clause in preparation of unit propagation.
- Examine the clause only when one of them is assigned to false.
- For each literal, maintain a watch list containing all the clauses that are composed by that literal.

Whenever a literal l is assigned to true, for each clause k that currently is watching or observing the negated literal $\neg l$, we do:

- If all but one literal l is assigned to false, assign l to true.
- If all literals are assigned to false, return the unsatisfiability of the problem.
- If any literal is assigned to true, continue.
- Otherwise, remove k from the watch list of $\neg l$ and add it to the watch list of one of its remaining unassigned literals.

Example

e.g. Watched literals mechanism.

Given the following clause:

$$c : x_1 \vee \neg x_2 \vee x_4 \vee x_4$$

We are gonna to introduce the main considerations about the watched literals behavior. We suppose that initially the approximated algorithm decided to observe x_1 and $\neg x_2$ literals from clause c.

- x_3 is assigned to false ($\neg x_3$), nothing happens, since the clause c is not in the watch list of the boolean variable x_3.
- x_2 is assign to true ($x_2 \rightarrow \neg x_2 \rightarrow$ false), therefore clause c is inspected. Since there is another literal to observe, we remove c from the watch list of $\neg x_2$ and we add the same clause c in the watch list of x_4.

The current situation is:

$$c : x_1 \vee \text{ false } \vee \text{ false } \vee x_4$$

both last literals are observed by the clause c.

- x_4 is assigned to false ($\neg x_4$). Clause c is again inspected, since it is already in the watch list of x_4, so we do unit propagation: all the literals of clause c are assigned to false, so we assign the last literal to true.
- x_1 is assigned to true, stating the satisfiability of the clause.

Chapter 7

SAT Encodings

Encoding in SAT can be **challenging**. If we are not careful about the encoding phase we may encounter big problems: we cannot directly **translate** any **Constraint Programming problem** into a **Satisfiability problem**, as we already did for N-Queens problem in the previous chapter.

The cardinality of the boolean decision variables used, as the number of clauses, can depend also from the dimension of the problem. An approach in which we have a total amount of variables and clauses similar to n^2 or worse is not a suitable approach for SAT problems. Not only the resolution of the problem might be very slow, but also the translation of the original propositional formula to its CNF form can be heavy.

7.1 Cardinality Constraints

Cardinality constraints are particular types of constraints often used in SAT models, where basically we have to control the summation over a set of boolean variables. Formally, they are defined as follows:

$$\sum_{1 \leq i \leq n} x_i \bowtie k$$

where:

- $x_i \rightarrow$ boolean variable
- $k \rightarrow$ fixed integer number
- $\bowtie \rightarrow$ comparison, it belongs to the set $\{<, \leq, =, \neq, >, \geq\}$

A special case of this definition is:

$$\sum_{1 \leq i \leq n} x_i = 1$$

that can be expressed also as:

$$\sum_{1 \leq i \leq n} x_i \leq 1 \wedge \sum_{1 \leq i \leq n} x_i \geq 1$$

The *exactly* condition can be described as: we want *at most one* variable set to 1 and at the same time we would like to obtain *at least one* single variable set to 1.

Now we write down a set of global constraints in order to describe such relationships, in particular:

1. **AtMostOne**$([x_1, x_2, \ldots, x_n])$, refers to

$$\sum_{1 \leq i \leq n} x_i \leq 1$$

2. **AtLeastOne**$([x_1, x_2, \ldots, x_n])$, refers to

$$\sum_{1 \leq i \leq n} x_i \geq 1$$

3. **ExactlyOne**$([x_1, x_2, \ldots, x_n])$, refers to

$$\textbf{AtMostOne}([x_1, x_2, \ldots, x_n]) \wedge \textbf{AtLeastOne}([x_1, x_2, \ldots, x_n])$$

It's very important to keep in mind such global constraints since they appear very often in logic problems:

1. N-Queens.
   - **Exactly one** queen on each row and column.
   - **At most one** queen on each diagonal.
2. Sudoku.
   - **Exactly one** presence of each number in each row, column and 3×3 grid.
   - **Exactly one** number in each cell.

However, how do we encode such constraints?

1. AtLeastOne$([x_1, x_2, \ldots, x_n])$
   A simple way for such constraint might be to insert a **logic or** between every couple of boolean variables, in particular, given the sequence $[x_1, x_2, \ldots, x_n]$ we should do:

$$x_1 \vee x_2 \vee \cdots \vee x_n$$

2. AtMostOne($[x_1, x_2, \ldots, x_n]$)
   AtMostOne($\ldots$) encoding constraint is not easy as before. There exist many feasible possibilities and the way we decide to encode it could affect the efficiency of the SAT solver.

7.2 AtMostOne: Pairwise Encoding

This is the obvious way to encode AtMostOne($\ldots$) constraint. The main idea is: any combination of two boolean variables cannot be **true** at the same time.

$$\bigwedge_{1 \leq j < i \leq n} \neg(x_i \wedge x_j)$$

where:

- $x_i \rightarrow$ boolean variable
- $x_j \rightarrow$ boolean variable

Therefore, for every pair $(i, j) \in [1, \ldots, n]$ we define this type of clause and between every of them will be a conjuction.

It's a very simple way to encode the constraint, without the usage of any additional variable. However, the total number of clauses is equal to $O(n^2)$, since we put this constraint for every pair of decision variables (i, j). As we already said, this type of complexity is not suitable for SAT problems.

7.3 AtMostOne: Sequential Encoding

Using the existing variables we cannot do better than $O(n^2)$, we need to use additional variables in our encoding. For sure, we improve the number of variables but at the same time we should reduce the cardinality of clauses.

In **sequential encoding** we introduce n new variables s_i used to indicate that the sum has reached 1 by i.

$$x_1 \rightarrow s_1 \wedge \bigwedge_{1 < i < n} [((x_i \vee s_{i-1})) \wedge (s_{i-1} \rightarrow \neg x_i)] \wedge (s_{n-1} \rightarrow \neg x_n)$$

s_i keeps track of what happened from the left-most variable of the sequence. Before moving on, let's take a more deep look to the previous formulation:

1. **First position**.
   If we reach 1 at the first position of the sequence then s_1 must be equal to 1.

2. **Middle positions**.
   In another position different from first one, if we have already reached 1 by the current boolean variables or before, s_i must be satisfied and, at the same time, all the other variables different from position $i-1$ must be set to false, 0.
3. **Last position**.
   Whether we have reached 1 then the last boolean variable x_n must be equal to false.

The SAT solver uses the CNF version of any propositional formula, therefore the previous formulation becomes:

$$(\neg x_1 \vee s_1) \wedge \bigwedge_{1<i<n} [(\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg s_{i-1} \vee \neg x_i)] \wedge (\neg s_{n-i} \vee \neg x_n)$$

where:

- $x_i \rightarrow$ boolean decision variable
- $s_i \rightarrow$ additional boolean variable to keep track of the assigned true values

Sequential encoding allows us to decrease the total amount of clauses. Overall the complexity of the variables does not really change; what changed it's the complexity of the cardinality of clauses, which from $O(n^2)$ becomes $O(n)$.

7.4 AtMostOne: Bitwise Encoding

Another way to encode AtMostOne(...) constraint is the **bitwise encoding**. It introduces m new variables r_i, where m is equal to $\log_2 n$. The additional variables r_i are unique signatures, each boolean variable x_i is mapped to an univocal r_i.

For $i \in [1, \dots, n]$, let $b_{i,1}, \dots, b_{i,m}$ be the sequence of binary numbers encoding a whatever boolean variable x_i, the bitwise formulation is:

$$\bigwedge_{1 \leq i \leq n} x_i \rightarrow (r_1 = b_{i,1} \wedge r_2 = b_{i,2} \wedge \cdots \wedge r_m = b_{i,m})$$

where:

- $x_i \rightarrow$ boolean variable
- $r_1, \dots, r_m \rightarrow$ unique signature used for each boolean variable
- $b_{1,m}, \dots, b_{i,m} \rightarrow$ boolean values or bits used to identify univocally the decision variable x_i

Every time x_i is equal to 1, we introduce other booleans to encode that variable x_i. Let's see an example for a better understanding.

Example

e.g. Consider the following equation.

$$x_1 + x_2 + x_3 \leq 1$$

The total number of variables is 3, therefore we have to introduce $\log_2(3) \simeq 2$ new r_i additional variables. Looking at what we said previously, the binary encoding is as follows:

$$\begin{aligned} &x_1 \rightarrow (r_1 = 0 \wedge r_2 = 0) \wedge \\ &x_2 \rightarrow (r_1 = 0 \wedge r_2 = 1) \wedge \\ &x_3 \rightarrow (r_1 = 1 \wedge r_2 = 0) \end{aligned}$$

In order to represent three variables we just need two bits ($2^2 = 4$ combinations to represent the boolean variables). Following this approach there's no way that more than one x_i is assigned to 1.

This is only the initial step, we know that the next thing to do is to translate the propositional formula to its CNF version, which becomes:

$$\begin{aligned} &(x_1 \vee \neg r_1) \wedge (x_1 \vee \neg r_2) \wedge \\ &(x_2 \vee \neg r_1) \wedge (x_2 \vee r_2) \wedge \\ &(x_3 \vee r_1) \wedge (x_3 \vee \neg r_2) \end{aligned}$$

For instance, if x_2 is equal to 1, then consequently our solver derive that $\neg r_1$ must be equal to false and r_2 must be equal to true, setting in this way the unique signature of the boolean variable x_2.

Moreover, if we suppose that both x_2 and x_3 are equal to 1, then r_1 additional variable would be false and true at the same time, causing a conflict. Therefore, the input data does not contain at most one single true value.

Generally, the CNF version of the bitwise encoding is:

$$\bigwedge_{1 \leq i \leq n} \bigwedge_{1 \leq j \leq m} \neg x_i \vee r_j [\vee \neg r_j]$$

The outer loop iterates over each variable x_i, while the inner loop looks at their signatures, where j defines the position of the bit.

Comparing the complexities, the bitwise paradigm as more clauses than sequential encoding, up to $O(n\log_2 n)$, but less additional variables, up to $O(\log_2 n)$.

7.5 AtMostOne: Heule Encoding

Heule encoding has the same complexities of **sequential encoding**, since we don't have a different cardinalities of clauses and variables. This paradigm is divided into two different resolutions, which are:

1. $n \leq 4$.
   When the number of variables is less or equal to four, we apply the **pairwise encoding**, using at most 6 clauses.

$$\bigwedge_{1 \leq i < j \leq n} \neg(x_i \wedge x_j)$$

2. $n > 4$.
   We split the sequence of boolean variables into two different sets. The first one contains the first three variables and a new one called y, while, in the other hand, the second set includes the negated version of y plus the remaining part of the original sequence.

$$\text{AtMostOne}([x_1, x_2, x_3, y]) \wedge \text{AtMostOne}([\neg y, x_4, \ldots, x_n])$$

   We keep doing the splitting until we reach the desired subdivsion.

Just to remind, the complexities about the clauses and variables are equal to $O(n)$.

7.6 Cardinality Constraints II

So far, we focused on $k = 1$, discarding a general integer value, and also we used only one specific comparison, $\leq$ (less or equal). As we stated, cardinality constraints are particular types of constraints used very often inside the SAT models, their formal definition is:

$$\sum_{1 \leq i \leq n} x_i \bowtie k$$

where:

- $x_i \rightarrow$ boolean variable
- $k \rightarrow$ fixed integer value
- $\bowtie \rightarrow$ comparison, it belongs to the set $\{<, \leq, =, \neq, >, \geq\}$

However, let's take a look to the whole picture considering also other comparisons, in particular:

- ...

As we can notice, every single comparison can be expressed as $\leq$ (less or equal) of a fixed integer number. Therefore, we could reduce our discussion to define the best way to encode the AtMostK(...) constraint, since it can be used for any other comparison.

Another special case is:

$$\sum_{1 \leq i \leq n} x_i = k \rightarrow \sum_{1 \leq i \leq n} x_i \leq k \wedge \sum_{1 \leq i \leq n} x_i \geq k$$

7.7 AtMostK: Generalized Pairwise Encoding

As before, we take a set of boolean variables and we state that they cannot be true at the same time. Formally, any combination of $k+1$ variables cannot be true at the same time.

$$\bigwedge_{\substack{M \subseteq \{1,\dots,n\} \\ |M| = k+1}} \bigvee_{i \in M} \neg x_i$$

We take a combination of $k+1$ variables since we want at most k boolean variables set to true, therefore any combination of $k+1$ variables cannot be possible.

This generalized approach is even more expensive than the original **pairwise encoding**, with a complexity up to $O(n^{k+1})$.

7.8 AtMostK: Sequential Encoding

The general idea was to introduce n new variables s_i to indicate that the sum has reached 1 by i, with a complexity proportional to n. With the AtMostK(...) constraint something changes, since we introduce an amount of $k * n$ additional variables $s_{i,j}$ to indicate that the sum has reached j by i. Basically, we use a 2 dimensional matrix in order to keep track which boolean variables have been set to true.

$$(x_1 \rightarrow s_{1,1}) \wedge \bigwedge_{2 \leq j \leq k} \overline{s_{1,j}}$$

$$\bigwedge_{1<i<n} \left[\left((x_i \vee s_{i-1,1}) \rightarrow s_{i,1}\right) \wedge \bigwedge_{2 \leq j \leq k} \left(\left((x_i \wedge s_{i-1,j-1}) \vee s_{i-1,j}\right) \rightarrow s_{i,j}\right) \wedge \left(s_{i-1,k} \rightarrow \overline{x_i}\right) \right]$$

$$\wedge \left(s_{n-1,k} \rightarrow \overline{x_n}\right)$$

where:

- $i \rightarrow$ index of boolean variables
- $j \rightarrow$ index to keep track the potential assignment of the variables to true
- $x_i \rightarrow$ boolean variables
- $s_{i,j} \rightarrow$ additional new boolean variables

Sequential encoding for constraint AtMostK(...) is very similar to the previous: its behavior follows the distinction in first position, middle positions and final position. Finally, its complexity is proportional to n, in real terms we have $O(kn)$ clauses and $O(kn)$ new variables.

7.9 SAT Encodings

There are different properties that we should consider when we are performing a SAT encoding. We have already claimed some of them, as the number of variables and the total cardinality of clauses. However, there is something more that we have to keep in mind while we are doing so, that is **constraint propagation**.

Let's consider an encoding E of a constraint C, such that there is a correspondence between the assignments of the variables in C with boolean assignments of the variables in E. E is arc-consistent if:

- Whenever a partial assignment is inconsistent wrt C (cannot be extended to a solution of C), unit propagation in E causes conflict.
- Otherwise, unit propagation in E discards arc-inconsistent values.

Example

e.g. Arc-Consistency with AtMostOne($[x_1, x_2, \ldots, x_n]$) constraint.

AtMostOne($[x_1, x_2, \ldots, x_n]$) is arc-consistent when:
- If there are two variables x_i and x_j assigned to T then unit propagation should give a conflict.
- If there is one x_i assigned to T then all the other x_j should be assigned to F by unit propagation.

Note: everything must be done by unit propagation, without any additional operation from the SAT solvers.

Are the AtMostOne(...) encodings we have seen so far arc-consistent encodings?
All of them are arc-consistent except one: **bitwise encoding**. Now we are gonna to see an example to state the inconsistency of this encoding, considering always unit

propagation as the main operation.

7.10 Addition in Propositional Logic

The example that we are gonna to consider is the **addition between binary numbers** in propositional logic. Formally, an addition in propositional logic is:

Decision problem

Given a and b represented in binary, find d satisfying $a + b = d$.

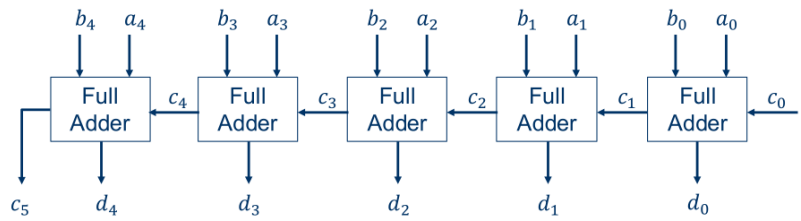
In SAT problems, we must use boolean variables to do so. Therefore, we list the following boolean variables:

- $a_{n-1}, \ldots, a_0 \rightarrow$ first binary number
- $b_{n-1}, \ldots, b_0 \rightarrow$ second binary number
- $d_{n-1}, \ldots, d_0 \rightarrow$ final result from the addition
- $c_n, c_{n-1}, \ldots, c_0 \rightarrow$ carries defined each time the sum between a, b and the previous carry c_{i-1} is greater than 2

Our goal is to compute each digit d_i of the result from the right to left starting from $c_0 = 0$, so the first carry is set to 0.

7.11 Full Adder

An interisting insight about the addition in propositional logic is given by the notion of **full adder**. We can think about the addition as a chain of operations, starting from the right-most digit. The full adder is the expression that takes in input the triple (a_i, b_i, c_i) and returns the couple (d_i, c_i).



The encoding for such chain of operations is defined as:

1st Result d.
We can state that d_i is equal to 1 if and only if the number of ones summed up is

odd, in other words:

$$d_i = a_i + b_i + c_i \bmod 2 \ \forall i \in \{0, \dots, n-1\}$$

The same notion can be represented as:

$$a_i \leftrightarrow b_i \leftrightarrow c_i \leftrightarrow d_i$$

that it can be expressed in CNF as:

$$d_i \leftrightarrow (a_i \wedge \neg b_i \wedge c_i) \vee (\neg a_i \wedge b_i \wedge c_i) \vee (\neg a_i \wedge \neg b_i \wedge c_i) \vee (a_i \wedge b_i \wedge c_i)$$

2nd Carries c.
Basically, every carry boolean variable is 1 if and only if the sum of the triple (a_i, b_i, c_{i-1}) is greater than 1.

$$c_{i+1} = 1 \leftrightarrow a_i + b_i + c_i > 1 \ \forall i \in \{0, \dots, n-1\}$$

As before, the same condition in CNF becomes:

$$c_{i+1} \leftrightarrow (a_i \wedge b_i) \vee (a_i \wedge c_i) \vee (b_i \wedge c_i)$$

3rd Carries c_n and c_0.
Both of them must be set to 0, in order to fit the whole sequence of bits and define the initial carry.

$$\neg c_0 \wedge \neg c_n$$

7.12 AtMostOne via Full Adder Encoding

Let's see how the **Full Adder Encoding** discussed previously can be used for the AtMostOne(...) constraint.

1. AtMostOne($[x_1, x_2]$) using one Full Adder.
   - a_0 and b_0 take values of x_1 and x_2.
   - Carries should be both set to 0, $\neg c_0 \wedge \neg c_1$
2. AtMostOne($[x_1, x_2, x_3]$) using one Full Adder.
   - a_0, b_0 and c_0 take values of x_1, x_2 and x_3.
   - Even in this case the carry c_1 should be set to 0, $\neg c_1$.
3. AtMostOne($[x_1, x_2, x_3, x_4]$) using two Full Adders.
   - a_0, b_0, a_1, b_1 take values of x_1, x_2, x_3 and x_4.

- We have to check carries and the sums of the d_i digits. The constraint is defined as follows:

$$\neg c_0 \wedge \neg c_1 \wedge \neg c_2 \wedge (\neg d_0 \vee \neg d_1)$$

  Forcing $\neg c_1$ and $\neg c_2$ we express that the sum should never exceed 0 or 1, at the same time the clause $(\neg d_0 \vee \neg d_1)$ defines that only one bit of the result can be set to 1.

7.13 AtMostK via Full Adder Encoding

Let's see the general case with k any fixed integer number.

AtMostK$([x_1, x_2, x_3, x_4], 2)$, so we would like to set up an encoding that allows us to check if the maximum number of bit set to 1 is exactly 2, using two Full Adders. As before, we distinguish boolean decision variables and the constraint related to them:

- a_0, b_0, a_1, b_1 take values of x_1, x_2, x_3 and x_4.
- We want at most two boolean variables set to 1, therefore the constraint is:

$$\neg c_0 \wedge (c_2 \rightarrow (\neg d_0 \wedge \neg d_1 \wedge \neg c_1))$$

  We are just performing the sum between binary numbers: starting from the notion $2_2 = 10$, we can use this information in order to describe our constraint. In this way, we know for sure that c_2 cannot be assigned to true and only d_0 or d_1 can be assigned to 1, providing that the final result does not exceed the fixed integer number.

However, considering $x_1 + x_2 + x_3 \leq 0$ we assume that by unit propagation the solver will set all these variables to false, without any additional operation from its list. As usual, we can exploit an encoding using one Full Adder:

- a_0, b_0 and c_0 take values of x_1, x_2 and x_3.
- In addition, our constraint is divided into:
  1. $\neg d_0 \wedge \neg c_1$
  2. $d_0 \leftrightarrow (x_1 \wedge \neg x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge x_3) \vee (x_1 \wedge x_2 \wedge x_3)$
  3. $c_1 \leftrightarrow (x_1 \wedge x_2) \vee (x_1 \wedge x_3) \vee (x_2 \wedge x_3)$
- Converting them into CNF form the whole thing becomes:
  2. $\neg d_0 \rightarrow (\neg x_1 \wedge x_2 \wedge x_3) \vee (x_1 \wedge \neg x_2 \wedge x_3) \vee (x_1 \wedge x_2 \wedge \neg x_3) \vee (\neg x_1 \wedge \neg x_2 \wedge \neg x_3)$
  3. $\neg c_1 \leftrightarrow (\neg x_1 \wedge \neg x_2) \vee (\neg x_1 \wedge \neg x_3) \vee (\neg x_2 \wedge \neg x_3)$

Unit propagation cannot propagate anything, since we have more than one free variable in both cases, for instance:

- $\neg d_0 \rightarrow (\neg x_1 \wedge x_2 \wedge x_3) \equiv \neg d_0 \vee (\neg x_1 \wedge x_2 \wedge x_3)$, 3 different free variables, which one should we propagate?
- $\neg c_1 \leftrightarrow (\neg x_1 \wedge \neg x_2) \equiv \neg c_1 \vee (\neg x_1 \wedge \neg x_2)$, 2 different free variables, which one should we propagate?

Therefore, without any additional operation, using exclusively unit propagation, our solver cannot derive that any boolean variable should be set to false. Adder encoding is not arc-consistent encoding (remember the two key points described in 7.9)!